Ganpat University

Department of Computer Science

M.SC(CA & IT) SEM - 1 Practical Documentation

Subject : MERN Stack Development

Academic Year : 2026-2027

Submitted by :

Name : Prince Prajapati

Enrolment Number : 26084231038

Practical – 01 : Build a React application using Functional Components, JSX and Component Reusability concepts.

1. Commands :

```
npm create vite@latest MERN_01
cd MERN_01
npm install
npm run dev
```

2. Project structure :

```
MERN_01/
|-- src/
|   |-- components/
|   |   |-- Header.jsx
|   |   |-- Button.jsx
|   |   |-- Card.jsx
|   |   |-- Badge.jsx
|   |   |-- CourseCard.jsx
|   |   |-- Footer.jsx
|   |-- App.jsx
|   |-- index.css
|   |-- main.jsx
|-- package.json
|-- index.html
```

3. Source Code :

- **src/App.jsx :**

```jsx
import React from 'react';
import Header from './components/Header';
import Footer from './components/Footer';
import Card from './components/Card';
import CourseCard from './components/CourseCard';
import Button from './components/Button';
import Badge from './components/Badge';

function App() {
  // Sample data to demonstrate component reusability with JSX and props
  const coreConcepts = [
    {
      id: 1,
      icon: '',
      title: 'Functional Components',
      description: 'JavaScript functions that accept props as an argument and return JSX. They
are simpler, lightweight, and modern standard in React.',
      category: 'React Core',
    },
    {
      id: 2,
      icon: '',
      title: 'JSX Syntax',
      description: 'JavaScript XML allows writing HTML-like syntax inside JavaScript, transpiled
to React.createElement() calls for high readability.',
      category: 'Syntax',
    },
    {
      id: 3,
```

```
      icon: '',
      title: 'Component Reusability',
      description: 'Building isolated, reusable UI blocks (Buttons, Cards, Badges) that receive
dynamic props to adapt across different application views.',
      category: 'Architecture',
    }
  ];

  const courses = [
    {
      id: 'CS101',
      courseCode: 'REACT-101',
      title: 'Modern React with JSX',
      instructor: 'Dr. John Doe',
      duration: '6 Weeks',
      level: 'Beginner',
      rating: 4.8
    },
    {
      id: 'CS102',
      courseCode: 'NODE-201',
      title: 'Backend Mastery with Express',
      instructor: 'Prof. Jane Smith',
      duration: '8 Weeks',
      level: 'Intermediate',
      rating: 4.9
    },
    {
      id: 'CS103',
      courseCode: 'MERN-301',
      title: 'Full Stack MERN Architecture',
      instructor: 'Alex Johnson',
      duration: '12 Weeks',
      level: 'Advanced',
      rating: 4.7
    }
  ];

  const handleEnroll = (courseTitle) => {
    alert(` Successfully enrolled in: ${courseTitle}`);
  };

  const handleLearnMore = (conceptTitle) => {
    alert(` Exploring details for: ${conceptTitle}`);
  };

  return (
    <div className="app-container">
      {/* 1. Header Functional Component */}
      <Header
        practicalNo="Practical No. 01"
        title="Functional Components, JSX & Reusability"
        subtitle="Demonstrating React Functional Component Architecture, JSX templating, and
reusable UI components"
      />

      <main className="main-content">
        {/* Concept Introduction Box */}
        <div className="concept-box">
          <h3> Practical Objective</h3>
```

```
          <p>
            Build a clean React application utilizing Functional Components, JSX expressions, and
modular
            Component Reusability. Reusable components like <code>Button</code>,
<code>Badge</code>, <code>Card</code>,
            and <code>CourseCard</code> are reused throughout the interface with varying props.
          </p>
        </div>

        {/* Section 1: Reusable Core Concept Cards */}
        <section className="section">
          <h2 className="section-title">
            <span>⚛</span> Core React Concepts (Reusable Card Component)
          </h2>
          <p className="section-desc">
            The same <code>&lt;Card /&gt;</code> component is reused 3 times with different props
data.
          </p>

          <div className="grid-3">
            {coreConcepts.map((item) => (
              <Card
                key={item.id}
                icon={item.icon}
                title={item.title}
                description={item.description}
                category={item.category}
                actionLabel="Explore Concept"
                onAction={() => handleLearnMore(item.title)}
              />
            ))}
          </div>
        </section>

        {/* Section 2: Reusable Course Cards */}
        <section className="section">
          <h2 className="section-title">
            <span>📚</span> Available Courses (Reusable CourseCard Component)
          </h2>
          <p className="section-desc">
            Demonstrating nested reusable components (<code>&lt;Badge /&gt;</code> and
<code>&lt;Button /&gt;</code> inside <code>&lt;CourseCard /&gt;</code>).
          </p>

          <div className="grid-3">
            {courses.map((course) => (
              <CourseCard
                key={course.id}
                courseCode={course.courseCode}
                title={course.title}
                instructor={course.instructor}
                duration={course.duration}
                level={course.level}
                rating={course.rating}
                onEnroll={handleEnroll}
              />
            ))}
          </div>
        </section>
```

```
        {/* Section 3: Standalone Reusable UI Elements */}
        <section className="section">
          <h2 className="section-title">
            <span>□</span> Reusable UI Buttons & Badges Showcase
          </h2>
          <p className="section-desc">
            Buttons and Badges rendering with different variant properties.
          </p>

          <div className="custom-card" style={{ gap: '1.25rem' }}>
            <div>
              <h4 style={{ marginBottom: '0.5rem', color: 'var(--text-muted)' }}>Button
Variants:</h4>
              <div style={{ display: 'flex', gap: '0.75rem', flexWrap: 'wrap' }}>
                <Button variant="primary" onClick={() => alert('Primary Clicked')}>Primary
Button</Button>
                <Button variant="secondary" onClick={() => alert('Secondary Clicked')}>Secondary
Button</Button>
                <Button variant="success" onClick={() => alert('Success Clicked')}>Success
Button</Button>
                <Button variant="primary" size="sm" onClick={() => alert('Small Clicked')}>Small
Button</Button>
              </div>
            </div>

            <div>
              <h4 style={{ marginBottom: '0.5rem', color: 'var(--text-muted)' }}>Badge
Variants:</h4>
              <div style={{ display: 'flex', gap: '0.75rem', flexWrap: 'wrap' }}>
                <Badge text="Primary Tag" variant="primary" />
                <Badge text="Active Status" variant="success" />
                <Badge text="Pending Review" variant="warning" />
                <Badge text="Informational" variant="info" />
              </div>
            </div>
          </div>
        </section>
      </main>

      {/* Footer Functional Component */}
      <Footer />
    </div>
  );
}

export default App;
```

- **src/components/Header.jsx :**

```
import React from 'react';

/**
 * Functional Component: Header
 * Demonstrates Functional component syntax, JSX and props usage.
 */
function Header({ title, subtitle, practicalNo }) {
  return (
```

```
    <header className="header">
      {practicalNo && <span className="header-badge">{practicalNo}</span>}
      <h1 className="header-title">{title}</h1>
      <p className="header-subtitle">{subtitle}</p>
    </header>
  );
}

export default Header;
```

- **src/components/Button.jsx :**

```
import React from 'react';

/**
 * Reusable Functional Component: Button
 * Reusable across different variants, sizes and click events.
 */
function Button({ children, variant = 'primary', size = 'md', onClick, icon }) {
  const sizeClass = size === 'sm' ? 'btn-sm' : '';
  const variantClass = `btn-${variant}`;

  return (
    <button className={`btn ${variantClass} ${sizeClass}`} onClick={onClick}>
      {icon && <span className="btn-icon">{icon}</span>}
      {children}
    </button>
  );
}

export default Button;
```

- **src/components/Card.jsx :**

```
import React from 'react';
import Badge from './Badge';
import Button from './Button';

/**
 * Reusable Functional Component: Feature Card
 * Demonstrates component nesting and JSX reusability
 */
function Card({ icon, title, description, category, actionLabel = "Learn More", onAction }) {
  return (
    <div className="custom-card">
      <div className="card-icon" style={{ background: '#eef2ff' }}>
        {icon}
      </div>
      <div style={{ marginBottom: '0.5rem' }}>
        <Badge text={category} variant="primary" />
      </div>
      <h3 className="card-title">{title}</h3>
      <p className="card-description">{description}</p>
      <div className="card-footer">
```

```
        <Button variant="primary" size="sm" onClick={onAction}>
          {actionLabel}
        </Button>
      </div>
    </div>
  );
}

export default Card;
```

- **src/components/Badge.jsx :**

```
import React from 'react';

/**
 * Reusable Functional Component: Badge
 * Demonstrates conditional class names and reusable tag rendering.
 */
function Badge({ text, variant = 'primary' }) {
  return (
    <span className={`badge badge-${variant}`}>
      {text}
    </span>
  );
}

export default Badge;
```

- **src/components/CourseCard.jsx :**

```
import React from 'react';
import Badge from './Badge';
import Button from './Button';

/**
 * Reusable Course Card component
 */
function CourseCard({ courseCode, title, instructor, duration, level, rating, onEnroll }) {
  const badgeVariant = level === 'Beginner' ? 'success' : level === 'Intermediate' ? 'info' :
'warning';

  return (
    <div className="custom-card">
      <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center',
marginBottom: '1rem' }}>
        <span style={{ fontSize: '0.85rem', fontWeight: '700', color: 'var(--primary)' }}>
          {courseCode}
        </span>
        <Badge text={level} variant={badgeVariant} />
      </div>

      <h3 className="card-title">{title}</h3>

      <div style={{ margin: '0.75rem 0', fontSize: '0.9rem', color: 'var(--text-muted)' }}>
```

```
        <p>👨‍🏫 <strong>Instructor:</strong> {instructor}</p>
        <p>⏱ <strong>Duration:</strong> {duration}</p>
        <p>⭐ <strong>Rating:</strong> {rating} / 5.0</p>
      </div>

      <div className="card-footer">
        <Button variant="success" size="sm" onClick={() => onEnroll(title)}>
          Enroll Now
        </Button>
        <Button variant="secondary" size="sm" onClick={() => alert(`Syllabus for ${title}`)}>
          View Syllabus
        </Button>
      </div>
    </div>
  );
}

export default CourseCard;
```

- **src/components/Footer.jsx :**

```
import React from 'react';

/**
 * Functional Component: Footer
 */
function Footer() {
  const currentYear = new Date().getFullYear();

  return (
    <footer className="footer">
      <p>
        <strong>Web Technology Practicals Journal</strong> | Built with React.js & Functional
Components
      </p>
      <p style={{ marginTop: '0.4rem', fontSize: '0.8rem', color: '#64748b' }}>
        Practical 1: Functional Components, JSX & Reusability &bull; &copy; {currentYear}
      </p>
    </footer>
  );
}

export default Footer;
```

- **src/index.css :**

```
:root {
  --primary: #6366f1;
  --primary-hover: #4f46e5;
  --primary-light: #e0e7ff;
  --secondary: #0ea5e9;
  --accent: #f59e0b;
  --success: #10b981;
  --danger: #ef4444;
```

```css
  --bg-main: #f8fafc;
  --bg-card: #ffffff;
  --text-main: #0f172a;
  --text-muted: #64748b;
  --border-color: #e2e8f0;
  --shadow-sm: 0 1px 2px 0 rgb(0 0 0 / 0.05);
  --shadow-md: 0 4px 6px -1px rgb(0 0 0 / 0.1), 0 2px 4px -2px rgb(0 0 0 / 0.1);
  --shadow-lg: 0 10px 15px -3px rgb(0 0 0 / 0.1), 0 4px 6px -4px rgb(0 0 0 / 0.1);
  --radius-md: 12px;
  --radius-lg: 16px;
  --font-family: 'Inter', system-ui, -apple-system, sans-serif;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: var(--font-family);
  background-color: var(--bg-main);
  color: var(--text-main);
  line-height: 1.6;
  -webkit-font-smoothing: antialiased;
}

.app-container {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

.main-content {
  flex: 1;
  max-width: 1200px;
  margin: 0 auto;
  padding: 2rem 1.5rem;
  width: 100%;
}

/* Header Component */
.header {
  background: linear-gradient(135deg, #1e1b4b 0%, #312e81 50%, #4338ca 100%);
  color: white;
  padding: 2.5rem 1.5rem;
  text-align: center;
  box-shadow: var(--shadow-md);
  position: relative;
  overflow: hidden;
}

.header::after {
  content: '';
  position: absolute;
  top: -50%;
  right: -20%;
  width: 300px;
  height: 300px;
  background: radial-gradient(circle, rgba(255,255,255,0.15) 0%, transparent 70%);
```

```
  border-radius: 50%;
}

.header-badge {
  display: inline-block;
  background: rgba(255, 255, 255, 0.2);
  backdrop-filter: blur(10px);
  padding: 0.35rem 1rem;
  border-radius: 9999px;
  font-size: 0.85rem;
  font-weight: 600;
  letter-spacing: 0.05em;
  text-transform: uppercase;
  margin-bottom: 0.75rem;
}

.header-title {
  font-size: 2.25rem;
  font-weight: 800;
  margin-bottom: 0.5rem;
  letter-spacing: -0.025em;
}

.header-subtitle {
  font-size: 1.1rem;
  color: #c7d2fe;
  max-width: 650px;
  margin: 0 auto;
}

/* Section styling */
.section {
  margin-bottom: 3rem;
}

.section-title {
  font-size: 1.5rem;
  font-weight: 700;
  color: var(--text-main);
  margin-bottom: 0.5rem;
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.section-desc {
  color: var(--text-muted);
  font-size: 0.95rem;
  margin-bottom: 1.5rem;
}

/* Reusable Grid */
.grid-3 {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(320px, 1fr));
  gap: 1.5rem;
}

.grid-2 {
  display: grid;
```

```
  grid-template-columns: repeat(auto-fit, minmax(450px, 1fr));
  gap: 1.5rem;
}

/* Reusable Card */
.custom-card {
  background: var(--bg-card);
  border: 1px solid var(--border-color);
  border-radius: var(--radius-lg);
  padding: 1.75rem;
  box-shadow: var(--shadow-sm);
  transition: transform 0.2s ease, box-shadow 0.2s ease;
  display: flex;
  flex-direction: column;
}

.custom-card:hover {
  transform: translateY(-4px);
  box-shadow: var(--shadow-lg);
}

.card-icon {
  width: 48px;
  height: 48px;
  border-radius: var(--radius-md);
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 1.5rem;
  margin-bottom: 1rem;
}

.card-title {
  font-size: 1.25rem;
  font-weight: 700;
  margin-bottom: 0.5rem;
  color: var(--text-main);
}

.card-description {
  color: var(--text-muted);
  font-size: 0.925rem;
  line-height: 1.5;
  margin-bottom: 1.25rem;
  flex: 1;
}

.card-footer {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-top: auto;
  padding-top: 1rem;
  border-top: 1px solid var(--border-color);
}

/* Reusable Button */
.btn {
  display: inline-flex;
  align-items: center;
```

```
  justify-content: center;
  gap: 0.5rem;
  font-family: inherit;
  font-size: 0.9rem;
  font-weight: 600;
  padding: 0.6rem 1.25rem;
  border-radius: 8px;
  border: none;
  cursor: pointer;
  transition: all 0.2s ease;
  text-decoration: none;
}

.btn-primary {
  background: var(--primary);
  color: white;
}
.btn-primary:hover {
  background: var(--primary-hover);
  box-shadow: 0 4px 12px rgba(99, 102, 241, 0.35);
}

.btn-secondary {
  background: #f1f5f9;
  color: #334155;
  border: 1px solid var(--border-color);
}
.btn-secondary:hover {
  background: #e2e8f0;
}

.btn-success {
  background: var(--success);
  color: white;
}
.btn-success:hover {
  background: #059669;
}

.btn-sm {
  padding: 0.4rem 0.85rem;
  font-size: 0.8rem;
}

/* Reusable Badge */
.badge {
  display: inline-flex;
  align-items: center;
  padding: 0.25rem 0.75rem;
  border-radius: 9999px;
  font-size: 0.75rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.05em;
}

.badge-primary {
  background: var(--primary-light);
  color: var(--primary);
}
```

```css
.badge-success {
  background: #d1fae5;
  color: #065f46;
}

.badge-warning {
  background: #fef3c7;
  color: #92400e;
}

.badge-info {
  background: #e0f2fe;
  color: #0369a1;
}

/* Info Box */
.concept-box {
  background: #ffffff;
  border-left: 4px solid var(--primary);
  border-radius: 0 var(--radius-md) var(--radius-md) 0;
  padding: 1.25rem 1.5rem;
  margin-bottom: 2rem;
  box-shadow: var(--shadow-sm);
}

.concept-box h3 {
  font-size: 1.1rem;
  color: var(--primary);
  margin-bottom: 0.5rem;
}

.concept-box p {
  color: var(--text-muted);
  font-size: 0.95rem;
}

/* Footer */
.footer {
  background: #0f172a;
  color: #94a3b8;
  padding: 2rem 1.5rem;
  text-align: center;
  font-size: 0.9rem;
  border-top: 1px solid #1e293b;
}

.footer strong {
  color: #f8fafc;
}
```

4. Output :

PRACTICAL NO. 01

Functional Components, JSX & Reusability

Demonstrating React Functional Component Architecture, JSX templating, and reusable UI components

Practical Objective

Build a clean React application utilizing Functional Components, JSX expressions, and modular Component Reusability. Reusable components like `Button`, `Badge`, `Card`, and `CourseCard` are reused throughout the interface with varying props.

Core React Concepts (Reusable Card Component)

The same `<Card />` component is reused 3 times with different props data.

REACT CORE

Functional Components

JavaScript functions that accept props as an argument and return JSX. They are simpler, lightweight, and modern standard in React.

Explore Concept

SYNTAX

JSX Syntax

JavaScript XML allows writing HTML-like syntax inside JavaScript, transpiled to React.createElement() calls for high readability.

Explore Concept

ARCHITECTURE

Component Reusability

Building isolated, reusable UI blocks (Buttons, Cards, Badges) that receive dynamic props to adapt across different application views.

Explore Concept

Practical – 02 : Develop a React Application implementing Props , State Management and Event Handling .

1. Commands :

```
npm create vite@latest MERN_02
cd MERN_02
npm install
npm run dev
```

2. Project structure :

```
MERN_02/
|-- src/
|   |-- components/
|   |   |-- Header.jsx
|   |   |-- CounterSection.jsx
|   |   |-- TaskManager.jsx
|   |   |-- TaskItem.jsx
|   |   |-- UserProfileCard.jsx
|   |   |-- InteractiveBox.jsx
|   |-- App.jsx
|   |-- index.css
|   |-- main.jsx
|-- package.json
|-- index.html
```

3. Source Code :

- **src/App.jsx :**

```jsx
import React from 'react';
import Header from './components/Header';
import CounterSection from './components/CounterSection';
import TaskManager from './components/TaskManager';
import UserProfileCard from './components/UserProfileCard';
import InteractiveBox from './components/InteractiveBox';

function App() {
  return (
    <div className="app-container">
      {/* Header */}
      <Header
        practicalNo="Practical No. 02"
        title="Props, State Management & Event Handling"
        subtitle="Practical implementation of Uni-directional Data Flow, useState Hook, and React Synthetic Event Handlers"
      />

      <main className="main-content">
        {/* Info Box */}
        <div style={{
          background: '#ffffff',
          borderLeft: '4px solid var(--primary)',
          padding: '1.25rem 1.5rem',
          borderRadius: '0 12px 12px 0',
          marginBottom: '2rem',
          boxShadow: '0 1px 3px rgba(0,0,0,0.05)'
        }}>
          <h3 style={{ fontSize: '1.1rem', color: 'var(--primary-dark)', marginBottom: '0.4rem'
```

```
}}>
            🎯 Practical Objective
          </h3>
          <p style={{ color: 'var(--text-muted)', fontSize: '0.95rem' }}>
            Develop a comprehensive React application demonstrating Props passing between parent-
child components,
            local State Management with the <code>useState</code> hook, and handling diverse
React Synthetic Events
            (<code>onClick</code>, <code>onChange</code>, <code>onSubmit</code>,
<code>onKeyDown</code>, <code>onMouseMove</code>).
          </p>
        </div>

        {/* Row 1: Interactive Counter & Task Manager */}
        <div className="grid-2">
          <CounterSection />
          <TaskManager />
        </div>

        {/* Row 2: Live Profile Customizer & Event Playground */}
        <div className="grid-2">
          <UserProfileCard />
          <InteractiveBox />
        </div>
      </main>

      {/* Footer */}
      <footer className="footer">
        <p><strong>Web Technology Practicals Journal</strong> &bull; Practical 02</p>
        <p style={{ marginTop: '0.35rem', fontSize: '0.8rem', color: '#64748b' }}>
          React Props, State Management & Event Handling System
        </p>
      </footer>
    </div>
  );
}

export default App;
```

- **src/components/CounterSection.jsx :**

```
import React, { useState } from 'react';

/**
 * Demonstrates:
 * 1. useState hook for managing numeric count and step size
 * 2. onClick event handling with increment, decrement, reset
 * 3. onChange event handling for dynamic step adjustments
 */
function CounterSection() {
  const [count, setCount] = useState(0);
  const [step, setStep] = useState(1);

  const handleIncrement = () => setCount(prev => prev + Number(step));
  const handleDecrement = () => setCount(prev => (prev - Number(step) >= 0 ? prev - Number(step)
: 0));
  const handleReset = () => setCount(0);
```

```
  const handleDouble = () => setCount(prev => prev * 2);

  return (
    <div className="card">
      <h3 className="card-title">
        <span>🔢</span> Interactive Counter (State & Click Events)
      </h3>
      <p className="card-subtitle">
        Demonstrates React <code>useState</code>, numeric state updates, and <code>onClick</code>
handlers.
      </p>

      <div className="counter-box">
        <div className="counter-display">{count}</div>

        <div style={{ width: '100%', maxWidth: '240px' }}>
          <label style={{ fontSize: '0.85rem', fontWeight: 600, color: 'var(--text-muted)' }}>
            Step Value:
          </label>
          <input
            type="number"
            min="1"
            max="100"
            value={step}
            onChange={(e) => setStep(Math.max(1, parseInt(e.target.value) || 1))}
            className="form-input"
            style={{ marginTop: '0.25rem', textAlign: 'center' }}
          />
        </div>

        <div className="counter-controls">
          <button className="btn btn-primary" onClick={handleIncrement}>
            +{step} Increment
          </button>
          <button className="btn btn-danger" onClick={handleDecrement}>
            -{step} Decrement
          </button>
          <button className="btn btn-warning" onClick={handleDouble}>
            ✖ 2 Double
          </button>
          <button className="btn btn-outline" onClick={handleReset}>
            ↺ Reset
          </button>
        </div>
      </div>
    </div>
  );
}

export default CounterSection;
```

- **src/components/TaskManager.jsx :**

```
import React, { useState } from 'react';
import TaskItem from './TaskItem';

/**
```

```jsx
 * TaskManager component
 * Demonstrates:
 * 1. Managing array state in React
 * 2. Form submission event (onSubmit & e.preventDefault())
 * 3. Passing functions as props down to child components
 * 4. Filtering state (All, Active, Completed)
 */
function TaskManager() {
  const [tasks, setTasks] = useState([
    { id: 1, text: 'Learn React Props & Component Hierarchy', completed: true },
    { id: 2, text: 'Master useState hook and functional updates', completed: false },
    { id: 3, text: 'Handle Synthetic Events in React', completed: false }
  ]);

  const [inputTask, setInputTask] = useState('');
  const [filter, setFilter] = useState('all');

  const handleAddTask = (e) => {
    e.preventDefault();
    if (!inputTask.trim()) return;

    const newTask = {
      id: Date.now(),
      text: inputTask.trim(),
      completed: false
    };

    setTasks([newTask, ...tasks]);
    setInputTask('');
  };

  const handleToggleTask = (id) => {
    setTasks(tasks.map(task =>
      task.id === id ? { ...task, completed: !task.completed } : task
    ));
  };

  const handleDeleteTask = (id) => {
    setTasks(tasks.filter(task => task.id !== id));
  };

  const filteredTasks = tasks.filter(task => {
    if (filter === 'active') return !task.completed;
    if (filter === 'completed') return task.completed;
    return true;
  });

  return (
    <div className="card">
      <h3 className="card-title">
        <span>🗒️</span> Task Manager (Props, List State & Callbacks)
      </h3>
      <p className="card-subtitle">
        Demonstrates parent-child state synchronization and callback prop patterns.
      </p>

      {/* Input Form with onSubmit */}
      <form onSubmit={handleAddTask} style={{ display: 'flex', gap: '0.5rem', marginBottom:
'1.25rem' }}>
        <input
```

```
          type="text"
          placeholder="Type a new task and press Add or Enter..."
          value={inputTask}
          onChange={(e) => setInputTask(e.target.value)}
          className="form-input"
        />
        <button type="submit" className="btn btn-primary" style={{ whiteSpace: 'nowrap' }}>
          ➕Add Task
        </button>
      </form>

      {/* Filter Tabs */}
      <div style={{ display: 'flex', gap: '0.5rem', marginBottom: '1rem' }}>
        <button
          className={`btn btn-sm ${filter === 'all' ? 'btn-primary' : 'btn-outline'}`}
          onClick={() => setFilter('all')}
        >
          All ({tasks.length})
        </button>
        <button
          className={`btn btn-sm ${filter === 'active' ? 'btn-primary' : 'btn-outline'}`}
          onClick={() => setFilter('active')}
        >
          Active ({tasks.filter(t => !t.completed).length})
        </button>
        <button
          className={`btn btn-sm ${filter === 'completed' ? 'btn-primary' : 'btn-outline'}`}
          onClick={() => setFilter('completed')}
        >
          Completed ({tasks.filter(t => t.completed).length})
        </button>
      </div>

      {/* Task List */}
      <div style={{ maxHeight: '250px', overflowY: 'auto' }}>
        {filteredTasks.length === 0 ? (
          <p style={{ textAlign: 'center', color: 'var(--text-muted)', padding: '1.5rem 0' }}>
            No tasks found in this view.
          </p>
        ) : (
          filteredTasks.map(task => (
            <TaskItem
              key={task.id}
              task={task}
              onToggle={handleToggleTask}
              onDelete={handleDeleteTask}
            />
          ))
        )}
      </div>
    </div>
  );
}

export default TaskManager;
```

- **src/components/UserProfileCard.jsx :**

```jsx
import React, { useState } from 'react';

/**
 * Demonstrates:
 * 1. Object state management in React
 * 2. Controlled form input handling with event target name & value
 * 3. Passing updated state values as Props to display sub-components
 */
function UserProfileCard() {
  const [profile, setProfile] = useState({
    name: 'Rahul Sharma',
    role: 'Full Stack Developer',
    experience: '3 Years',
    location: 'Mumbai, India',
    isAvailable: true
  });

  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    setProfile(prev => ({
      ...prev,
      [name]: type === 'checkbox' ? checked : value
    }));
  };

  return (
    <div className="card">
      <h3 className="card-title">
        <span>□</span> Live User Profile (Object State & Change Events)
      </h3>
      <p className="card-subtitle">
        Update the inputs below to see immediate two-way state reflection in the profile card.
      </p>

      {/* Form Controls */}
      <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '0.75rem',
marginBottom: '1.25rem' }}>
        <div>
          <label style={{ fontSize: '0.8rem', fontWeight: 600, color: 'var(--text-muted)'
}}>Name:</label>
          <input
            type="text"
            name="name"
            value={profile.name}
            onChange={handleChange}
            className="form-input"
          />
        </div>
        <div>
          <label style={{ fontSize: '0.8rem', fontWeight: 600, color: 'var(--text-muted)'
}}>Role:</label>
          <input
            type="text"
            name="role"
            value={profile.role}
            onChange={handleChange}
            className="form-input"
          />
        </div>
        <div>
```

```
          <label style={{ fontSize: '0.8rem', fontWeight: 600, color: 'var(--text-muted)'
}}>Experience:</label>
          <input
            type="text"
            name="experience"
            value={profile.experience}
            onChange={handleChange}
            className="form-input"
          />
        </div>
        <div>
          <label style={{ fontSize: '0.8rem', fontWeight: 600, color: 'var(--text-muted)'
}}>Location:</label>
          <input
            type="text"
            name="location"
            value={profile.location}
            onChange={handleChange}
            className="form-input"
          />
        </div>
      </div>

      <div style={{ marginBottom: '1.25rem' }}>
        <label style={{ display: 'inline-flex', alignItems: 'center', gap: '0.5rem', cursor:
'pointer', fontSize: '0.9rem', fontWeight: 600 }}>
          <input
            type="checkbox"
            name="isAvailable"
            checked={profile.isAvailable}
            onChange={handleChange}
            className="task-checkbox"
          />
          Available for Freelance / Hire
        </label>
      </div>

      {/* Live Preview Card */}
      <div className="user-preview">
        <div className="user-avatar">
          {profile.name ? profile.name.charAt(0).toUpperCase() : '?'}
        </div>
        <div>
          <h4 style={{ fontSize: '1.15rem', fontWeight: 700 }}>{profile.name || 'Anonymous'}</h4>
          <p style={{ color: 'var(--primary-dark)', fontWeight: 600, fontSize: '0.9rem'
}}>{profile.role || 'No Role'}</p>
          <p style={{ color: 'var(--text-muted)', fontSize: '0.85rem' }}>
            □ {profile.location} &bull; □ {profile.experience}
          </p>
          <span style={{
            display: 'inline-block',
            marginTop: '0.35rem',
            padding: '0.2rem 0.6rem',
            borderRadius: '9999px',
            fontSize: '0.75rem',
            fontWeight: 700,
            background: profile.isAvailable ? '#dcfce7' : '#fee2e2',
            color: profile.isAvailable ? '#15803d' : '#b91c1c'
          }}>
            {profile.isAvailable ? '● Available' : '○ Unavailable'}
```

```
          </span>
        </div>
      </div>
    </div>
  );
}

export default UserProfileCard;
```

- **src/components/InteractiveBox.jsx :**

```
import React, { useState } from 'react';

/**
 * Demonstrates:
 * 1. Mouse Events: onMouseEnter, onMouseLeave, onMouseMove
 * 2. Keyboard Events: onKeyDown
 * 3. Dynamic styling based on component state
 */
function InteractiveBox() {
  const [bgChoice, setBgChoice] = useState('indigo');
  const [isHovered, setIsHovered] = useState(false);
  const [lastKeyPressed, setLastKeyPressed] = useState('None');
  const [mouseCoord, setMouseCoord] = useState({ x: 0, y: 0 });

  const colorPalettes = {
    indigo: 'linear-gradient(135deg, #6366f1 0%, #4338ca 100%)',
    emerald: 'linear-gradient(135deg, #10b981 0%, #047857 100%)',
    rose: 'linear-gradient(135deg, #f43f5e 0%, #be123c 100%)',
    amber: 'linear-gradient(135deg, #f59e0b 0%, #b45309 100%)'
  };

  const handleMouseMove = (e) => {
    const rect = e.currentTarget.getBoundingClientRect();
    setMouseCoord({
      x: Math.round(e.clientX - rect.left),
      y: Math.round(e.clientY - rect.top)
    });
  };

  return (
    <div className="card">
      <h3 className="card-title">
        <span>⚡</span> Mouse & Keyboard Events Playground
      </h3>
      <p className="card-subtitle">
        Hover, move cursor inside the canvas, and press keys to inspect synthetic event payload.
      </p>

      {/* Color theme selectors */}
      <div style={{ display: 'flex', gap: '0.5rem', marginBottom: '1rem', alignItems: 'center'
}}>
        <span style={{ fontSize: '0.85rem', fontWeight: 600 }}>Palette:</span>
        {Object.keys(colorPalettes).map((color) => (
          <button
            key={color}
            onClick={() => setBgChoice(color)}
```

```
            style={{
              padding: '0.35rem 0.75rem',
              borderRadius: '6px',
              border: 'none',
              cursor: 'pointer',
              fontWeight: 600,
              fontSize: '0.8rem',
              textTransform: 'capitalize',
              background: bgChoice === color ? '#0f172a' : '#e2e8f0',
              color: bgChoice === color ? '#ffffff' : '#334155'
            }}
          >
            {color}
          </button>
        ))}
      </div>

      {/* Interactive canvas box */}
      <div
        onMouseEnter={() => setIsHovered(true)}
        onMouseLeave={() => setIsHovered(false)}
        onMouseMove={handleMouseMove}
        style={{
          background: colorPalettes[bgChoice],
          color: 'white',
          borderRadius: '12px',
          padding: '2rem 1.5rem',
          textAlign: 'center',
          transition: 'transform 0.2s ease, box-shadow 0.2s ease',
          transform: isHovered ? 'scale(1.02)' : 'scale(1)',
          boxShadow: isHovered ? '0 10px 25px rgba(0,0,0,0.2)' : 'none',
          cursor: 'crosshair',
          marginBottom: '1rem'
        }}
      >
        <h4 style={{ fontSize: '1.25rem', marginBottom: '0.5rem' }}>
          {isHovered ? '✨Mouse is Inside Box!' : '🖱️ Hover Cursor Here'}
        </h4>
        <p style={{ fontSize: '0.9rem', opacity: 0.9 }}>
          Cursor Coordinates: <strong>X: {mouseCoord.x}px | Y: {mouseCoord.y}px</strong>
        </p>
      </div>

      {/* Keydown testing input */}
      <div>
        <label style={{ fontSize: '0.85rem', fontWeight: 600, color: 'var(--text-muted)' }}>
          Test Keyboard Event (Type any key inside input):
        </label>
        <input
          type="text"
          placeholder="Press any key here..."
          onKeyDown={(e) => setLastKeyPressed(`${e.key} (Code: ${e.code})`)}
          className="form-input"
          style={{ marginTop: '0.25rem' }}
        />
        <p style={{ marginTop: '0.4rem', fontSize: '0.85rem', color: 'var(--text-muted)' }}>
          Last Key Detected: <strong style={{ color: 'var(--primary-dark)'
}}>{lastKeyPressed}</strong>
        </p>
      </div>
```

```
    </div>
  );
}

export default InteractiveBox;
```

- **src/index.css :**

```
:root {
  --primary: #0284c7;
  --primary-dark: #0369a1;
  --primary-light: #e0f2fe;
  --accent: #8b5cf6;
  --success: #10b981;
  --danger: #ef4444;
  --warning: #f59e0b;
  --bg-main: #f8fafc;
  --bg-card: #ffffff;
  --text-main: #0f172a;
  --text-muted: #64748b;
  --border-color: #e2e8f0;
  --shadow-sm: 0 1px 3px 0 rgb(0 0 0 / 0.1);
  --shadow-md: 0 4px 6px -1px rgb(0 0 0 / 0.1), 0 2px 4px -2px rgb(0 0 0 / 0.1);
  --shadow-lg: 0 10px 15px -3px rgb(0 0 0 / 0.1);
  --radius: 12px;
  --font-family: 'Plus Jakarta Sans', system-ui, -apple-system, sans-serif;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: var(--font-family);
  background-color: var(--bg-main);
  color: var(--text-main);
  line-height: 1.6;
}

.app-container {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

.main-content {
  flex: 1;
  max-width: 1200px;
  margin: 0 auto;
  padding: 2rem 1.5rem;
  width: 100%;
}

.header {
  background: linear-gradient(135deg, #0c4a6e 0%, #0369a1 50%, #0284c7 100%);
```

```
  color: white;
  padding: 2.5rem 1.5rem;
  text-align: center;
  box-shadow: var(--shadow-md);
}

.header-badge {
  display: inline-block;
  background: rgba(255, 255, 255, 0.2);
  backdrop-filter: blur(8px);
  padding: 0.35rem 1rem;
  border-radius: 9999px;
  font-size: 0.85rem;
  font-weight: 700;
  text-transform: uppercase;
  margin-bottom: 0.75rem;
}

.header-title {
  font-size: 2.25rem;
  font-weight: 800;
  margin-bottom: 0.5rem;
}

.header-subtitle {
  font-size: 1.05rem;
  color: #bae6fd;
  max-width: 650px;
  margin: 0 auto;
}

.grid-2 {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(400px, 1fr));
  gap: 1.75rem;
  margin-bottom: 2rem;
}

.card {
  background: var(--bg-card);
  border: 1px solid var(--border-color);
  border-radius: var(--radius);
  padding: 1.75rem;
  box-shadow: var(--shadow-sm);
}

.card-title {
  font-size: 1.25rem;
  font-weight: 700;
  color: var(--text-main);
  margin-bottom: 0.5rem;
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

.card-subtitle {
  font-size: 0.9rem;
  color: var(--text-muted);
  margin-bottom: 1.5rem;
```

```
}

/* Counter styles */
.counter-box {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 1.25rem;
  padding: 1.5rem;
  background: #f0f9ff;
  border: 1px dashed var(--primary);
  border-radius: var(--radius);
}

.counter-display {
  font-size: 3.5rem;
  font-weight: 800;
  color: var(--primary-dark);
  line-height: 1;
}

.counter-controls {
  display: flex;
  gap: 0.75rem;
  flex-wrap: wrap;
  justify-content: center;
}

/* Button UI */
.btn {
  padding: 0.6rem 1.25rem;
  font-size: 0.9rem;
  font-weight: 600;
  border-radius: 8px;
  border: none;
  cursor: pointer;
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
  transition: all 0.2s ease;
  font-family: inherit;
}

.btn-primary { background: var(--primary); color: white; }
.btn-primary:hover { background: var(--primary-dark); }
.btn-success { background: var(--success); color: white; }
.btn-success:hover { background: #059669; }
.btn-danger { background: var(--danger); color: white; }
.btn-danger:hover { background: #dc2626; }
.btn-warning { background: var(--warning); color: white; }
.btn-warning:hover { background: #d97706; }
.btn-outline { background: transparent; border: 1px solid var(--border-color); color: var(--text-
main); }
.btn-outline:hover { background: #f1f5f9; }
.btn-sm { padding: 0.35rem 0.75rem; font-size: 0.8rem; }

/* Input controls */
.form-input {
  width: 100%;
  padding: 0.65rem 0.9rem;
```

```
  font-size: 0.95rem;
  border: 1px solid var(--border-color);
  border-radius: 8px;
  outline: none;
  font-family: inherit;
  transition: border-color 0.2s;
}

.form-input:focus {
  border-color: var(--primary);
  box-shadow: 0 0 0 3px rgba(2, 132, 199, 0.15);
}

/* Task list styles */
.task-item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0.85rem 1rem;
  background: #f8fafc;
  border: 1px solid var(--border-color);
  border-radius: 8px;
  margin-bottom: 0.6rem;
  transition: all 0.2s ease;
}

.task-item.completed {
  background: #f1f5f9;
  opacity: 0.7;
}

.task-item.completed .task-text {
  text-decoration: line-through;
  color: var(--text-muted);
}

.task-left {
  display: flex;
  align-items: center;
  gap: 0.75rem;
}

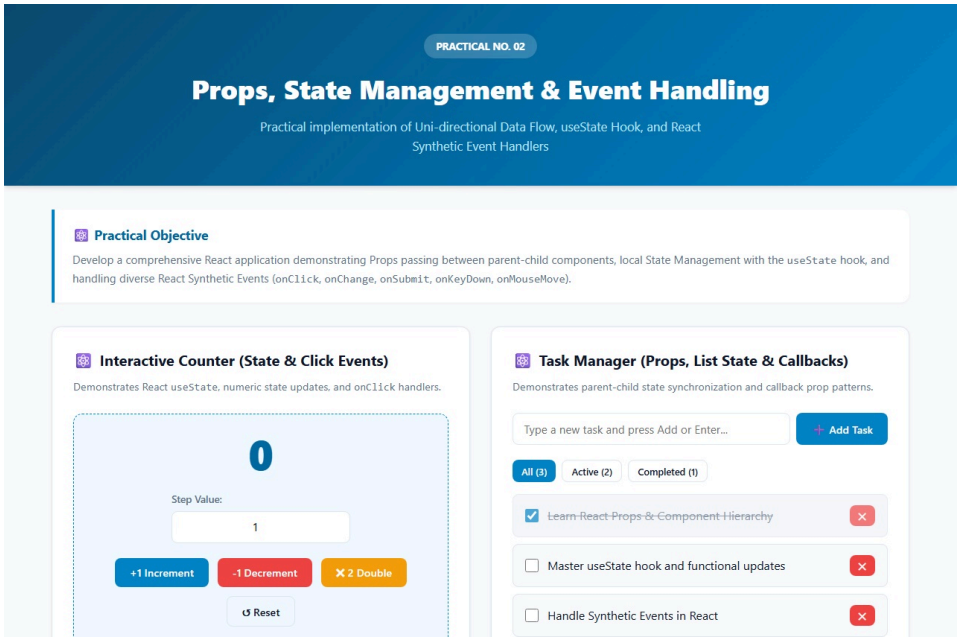
.task-checkbox {
  width: 18px;
  height: 18px;
  cursor: pointer;
  accent-color: var(--primary);
}

/* User Card Styles */
.user-preview {
  display: flex;
  gap: 1.25rem;
  align-items: center;
  padding: 1.25rem;
  border-radius: var(--radius);
  background: linear-gradient(135deg, #f0f9ff 0%, #e0f2fe 100%);
  border: 1px solid #bae6fd;
}
```

```css
.user-avatar {
  width: 64px;
  height: 64px;
  border-radius: 50%;
  background: var(--primary);
  color: white;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 1.75rem;
  font-weight: 700;
  box-shadow: var(--shadow-sm);
}

.footer {
  background: #0f172a;
  color: #94a3b8;
  padding: 2rem 1.5rem;
  text-align: center;
  font-size: 0.9rem;
  border-top: 1px solid #1e293b;
}
```

4. Output :



Practical – 03 : Design a Student Management Form with Controlled Components, Multi-field Regex Validations, and Local Storage CRUD.

1. Commands :

```
npm create vite@latest MERN_03
cd MERN_03
npm install
npm run dev
```

2. Project structure :

```
MERN_03/
|-- src/
|   |-- components/
|   |   |-- Header.jsx
|   |   |-- StudentForm.jsx
|   |   |-- StudentTable.jsx
|   |   |-- StatsOverview.jsx
|   |-- utils/
|   |   |-- validation.js
|   |-- App.jsx
|   |-- index.css
|   |-- main.jsx
|-- package.json
|-- index.html
```

3. Source Code :

- **src/App.jsx :**

```jsx
import React, { useState } from 'react';
import Header from './components/Header';
import StatsOverview from './components/StatsOverview';
import StudentForm from './components/StudentForm';
import StudentTable from './components/StudentTable';

const INITIAL_STUDENTS = [
  {
    id: 'STU-1001',
    fullName: 'Prince Prajapati'
    rollNo: '26084231038'
    email: 'prince@student.edu'
    phone: '9876543210',
    department: 'Computer Engineering',
    semester: 'Semester 6',
    gender: 'Male',
    skills: ['React', 'Node.js', 'MongoDB', 'JavaScript'],
    address: 'Ahmedabad, Gujarat'
  },
  {
    id: 'STU-1002',
    fullName: 'Priya Sharma',
    rollNo: '21IT012',
    email: 'priya.sharma@student.edu',
    phone: '9812345678',
    department: 'Information Technology',
    semester: 'Semester 6',
    gender: 'Female',
    skills: ['Python', 'SQL', 'React'],
```

```
    address: 'Mumbai, Maharashtra'
  },
  {
    id: 'STU-1003',
    fullName: 'Aman Verma',
    rollNo: '22AI088',
    email: 'aman.verma@student.edu',
    phone: '9765432109',
    department: 'AI & Data Science',
    semester: 'Semester 4',
    gender: 'Male',
    skills: ['Python', 'Java', 'SQL'],
    address: 'Pune, Maharashtra'
  }
];

function App() {
  const [students, setStudents] = useState(INITIAL_STUDENTS);
  const [editingStudent, setEditingStudent] = useState(null);
  const [searchQuery, setSearchQuery] = useState('');
  const [selectedDeptFilter, setSelectedDeptFilter] = useState('All');
  const [alertMessage, setAlertMessage] = useState(null);

  const showAlert = (text, type = 'success') => {
    setAlertMessage({ text, type });
    setTimeout(() => setAlertMessage(null), 3500);
  };

  // Add or Update student handler
  const handleSaveStudent = (studentData) => {
    if (editingStudent) {
      setStudents(prev => prev.map(s => s.id === editingStudent.id ? { ...studentData, id: s.id }
: s));
      showAlert(`✅Student record "${studentData.fullName}" updated successfully!`);
      setEditingStudent(null);
    } else {
      const newStudent = {
        ...studentData,
        id: `STU-${Date.now().toString().slice(-4)}`
      };
      setStudents(prev => [newStudent, ...prev]);
      showAlert(`🎉 Student "${studentData.fullName}" added successfully!`);
    }
  };

  // Delete student handler
  const handleDeleteStudent = (id, name) => {
    if (window.confirm(`Are you sure you want to delete student: ${name}?`)) {
      setStudents(prev => prev.filter(s => s.id !== id));
      if (editingStudent && editingStudent.id === id) {
        setEditingStudent(null);
      }
      showAlert(`🗑️ Student record "${name}" deleted.`, 'danger');
    }
  };

  // Edit initiate
  const handleEditStudent = (student) => {
    setEditingStudent(student);
    window.scrollTo({ top: 150, behavior: 'smooth' });
```

```
  };

  // Filter students based on search and department
  const filteredStudents = students.filter(student => {
    const matchesSearch =
      student.fullName.toLowerCase().includes(searchQuery.toLowerCase()) ||
      student.rollNo.toLowerCase().includes(searchQuery.toLowerCase()) ||
      student.email.toLowerCase().includes(searchQuery.toLowerCase());

    const matchesDept = selectedDeptFilter === 'All' || student.department ===
selectedDeptFilter;

    return matchesSearch && matchesDept;
  });

  return (
    <div className="app-container">
      {/* Header */}
      <Header
        practicalNo="Practical No. 03"
        title="Student Management System"
        subtitle="Comprehensive React Forms, Controlled Components, Live Field Validations & Full
CRUD Operations"
      />

      <main className="main-content">
        {/* Toast Alert Notification */}
        {alertMessage && (
          <div style={{
            background: alertMessage.type === 'danger' ? '#fee2e2' : '#dcfce7',
            color: alertMessage.type === 'danger' ? '#991b1b' : '#166534',
            border: `1px solid ${alertMessage.type === 'danger' ? '#f87171' : '#86efac'}`,
            padding: '0.85rem 1.25rem',
            borderRadius: '8px',
            marginBottom: '1.5rem',
            fontWeight: 600,
            fontSize: '0.9rem',
            boxShadow: '0 2px 4px rgba(0,0,0,0.05)'
          }}>
            {alertMessage.text}
          </div>
        )}

        {/* Top Stats Overview */}
        <StatsOverview students={students} />

        {/* 2-Column Responsive Layout: Form on Left, Table on Right */}
        <div className="layout-grid">
          {/* Left Column: Form with Controlled Components */}
          <StudentForm
            onSaveStudent={handleSaveStudent}
            editingStudent={editingStudent}
            onCancelEdit={() => setEditingStudent(null)}
            existingStudents={students}
          />

          {/* Right Column: Search, Filter & Student Records Table */}
          <div className="card">
            <div className="card-title">
              <span>📋 Registered Student Directory</span>
```

```
              <span style={{ fontSize: '0.85rem', color: 'var(--text-muted)' }}>
                Showing {filteredStudents.length} of {students.length}
              </span>
            </div>

            {/* Filter and Search Bar */}
            <div style={{ display: 'grid', gridTemplateColumns: '1fr auto', gap: '0.75rem',
marginBottom: '1.25rem' }}>
              <input
                type="text"
                placeholder="🔍 Search by name, roll no, email..."
                value={searchQuery}
                onChange={(e) => setSearchQuery(e.target.value)}
                className="form-input"
              />
              <select
                value={selectedDeptFilter}
                onChange={(e) => setSelectedDeptFilter(e.target.value)}
                className="form-select"
                style={{ width: 'auto', minWidth: '180px' }}
              >
                <option value="All">All Departments</option>
                <option value="Computer Engineering">Computer Engineering</option>
                <option value="Information Technology">Information Technology</option>
                <option value="AI & Data Science">AI & Data Science</option>
                <option value="Electronics & Comm">Electronics & Comm</option>
                <option value="Mechanical Engineering">Mechanical Engineering</option>
              </select>
            </div>

            {/* Table */}
            <StudentTable
              students={filteredStudents}
              onEdit={handleEditStudent}
              onDelete={handleDeleteStudent}
            />
          </div>
        </div>
      </main>

      {/* Footer */}
      <footer className="footer">
        <p><strong>Web Technology Practicals Journal</strong> &bull; Practical 03</p>
        <p style={{ marginTop: '0.35rem', fontSize: '0.8rem', color: '#64748b' }}>
          Student Management System using React Forms, Controlled Components and Validation
        </p>
      </footer>
    </div>
  );
}

export default App;
```

- **src/components/StudentForm.jsx :**

```
import React, { useState, useEffect } from 'react';
import { validateStudentForm } from '../utils/validation';
```

```
const initialFormState = {
  fullName: '',
  rollNo: '',
  email: '',
  phone: '',
  department: '',
  semester: '',
  gender: '',
  skills: [],
  address: ''
};

const AVAILABLE_SKILLS = ['JavaScript', 'React', 'Node.js', 'Python', 'MongoDB', 'SQL', 'Java'];

function StudentForm({ onSaveStudent, editingStudent, onCancelEdit, existingStudents }) {
  const [formData, setFormData] = useState(initialFormState);
  const [errors, setErrors] = useState({});
  const [touched, setTouched] = useState({});

  useEffect(() => {
    if (editingStudent) {
      setFormData(editingStudent);
      setErrors({});
      setTouched({});
    } else {
      setFormData(initialFormState);
      setErrors({});
      setTouched({});
    }
  }, [editingStudent]);

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData(prev => ({ ...prev, [name]: value }));

    // Clear error for this field as user types
    if (errors[name]) {
      setErrors(prev => ({ ...prev, [name]: '' }));
    }
  };

  const handleBlur = (e) => {
    const { name } = e.target;
    setTouched(prev => ({ ...prev, [name]: true }));
    const { errors: newErrors } = validateStudentForm(formData, existingStudents, editingStudent
? editingStudent.id : null);
    if (newErrors[name]) {
      setErrors(prev => ({ ...prev, [name]: newErrors[name] }));
    }
  };

  const handleSkillToggle = (skill) => {
    setFormData(prev => {
      const skills = prev.skills.includes(skill)
        ? prev.skills.filter(s => s !== skill)
        : [...prev.skills, skill];
      return { ...prev, skills };
    });
    if (errors.skills) {
```

```
      setErrors(prev => ({ ...prev, skills: '' }));
    }
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    const { isValid, errors: validationErrors } = validateStudentForm(
      formData,
      existingStudents,
      editingStudent ? editingStudent.id : null
    );

    if (!isValid) {
      setErrors(validationErrors);
      // Mark all fields as touched to display errors
      const allTouched = Object.keys(formData).reduce((acc, key) => ({ ...acc, [key]: true }),
{});
      setTouched(allTouched);
      return;
    }

    onSaveStudent(formData);
    setFormData(initialFormState);
    setErrors({});
    setTouched({});
  };

  return (
    <div className="card">
      <div className="card-title">
        <span>{editingStudent ? '✏️ Edit Student Record' : '➕ Add New Student'}</span>
        {editingStudent && (
          <span style={{ fontSize: '0.8rem', color: 'var(--primary-dark)', fontWeight: 600 }}>
            Editing ID: {editingStudent.id}
          </span>
        )}
      </div>

      <form onSubmit={handleSubmit} noValidate>
        {/* Full Name */}
        <div className="form-group">
          <label className="form-label">Full Name *</label>
          <input
            type="text"
            name="fullName"
            placeholder="e.g. Prince Prajapati
            value={formData.fullName}
            onChange={handleChange}
            onBlur={handleBlur}
            className={`form-input ${touched.fullName && errors.fullName ? 'has-error' : ''}`}
          />
          {touched.fullName && errors.fullName && <p className="error-text">⚠
{errors.fullName}</p>}
        </div>

        {/* Roll Number & Email in 2 columns */}
        <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '0.75rem' }}>
          <div className="form-group">
            <label className="form-label">Enrollment / Roll No *</label>
            <input
```

```
              type="text"
              name="rollNo"
              placeholder="e.g. 26084231038
              value={formData.rollNo}
              onChange={handleChange}
              onBlur={handleBlur}
              className={`form-input ${touched.rollNo && errors.rollNo ? 'has-error' : ''}`}
            />
            {touched.rollNo && errors.rollNo && <p className="error-text">⚠ {errors.rollNo}</p>}
          </div>

          <div className="form-group">
            <label className="form-label">Email Address *</label>
            <input
              type="email"
              name="email"
              placeholder="e.g. student@college.edu"
              value={formData.email}
              onChange={handleChange}
              onBlur={handleBlur}
              className={`form-input ${touched.email && errors.email ? 'has-error' : ''}`}
            />
            {touched.email && errors.email && <p className="error-text">⚠ {errors.email}</p>}
          </div>
        </div>

        {/* Phone & Department */}
        <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '0.75rem' }}>
          <div className="form-group">
            <label className="form-label">Phone Number *</label>
            <input
              type="tel"
              name="phone"
              maxLength="10"
              placeholder="10-digit Mobile"
              value={formData.phone}
              onChange={handleChange}
              onBlur={handleBlur}
              className={`form-input ${touched.phone && errors.phone ? 'has-error' : ''}`}
            />
            {touched.phone && errors.phone && <p className="error-text">⚠ {errors.phone}</p>}
          </div>

          <div className="form-group">
            <label className="form-label">Department / Branch *</label>
            <select
              name="department"
              value={formData.department}
              onChange={handleChange}
              onBlur={handleBlur}
              className={`form-select ${touched.department && errors.department ? 'has-error' :
''}`}
            >
              <option value="">-- Select Branch --</option>
              <option value="Computer Engineering">Computer Engineering</option>
              <option value="Information Technology">Information Technology</option>
              <option value="AI & Data Science">AI & Data Science</option>
              <option value="Electronics & Comm">Electronics & Comm</option>
              <option value="Mechanical Engineering">Mechanical Engineering</option>
            </select>
```

```
            {touched.department && errors.department && <p className="error-text">⚠ {errors.department}</p>}
          </div>
        </div>

        {/* Semester & Gender */}
        <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '0.75rem' }}>
          <div className="form-group">
            <label className="form-label">Current Semester *</label>
            <select
              name="semester"
              value={formData.semester}
              onChange={handleChange}
              onBlur={handleBlur}
              className={`form-select ${touched.semester && errors.semester ? 'has-error' : ''}`}
            >
              <option value="">-- Select Sem --</option>
              {[1, 2, 3, 4, 5, 6, 7, 8].map(s => (
                <option key={s} value={`Semester ${s}`}>Semester {s}</option>
              ))}
            </select>
            {touched.semester && errors.semester && <p className="error-text">⚠ {errors.semester}</p>}
          </div>

          <div className="form-group">
            <label className="form-label">Gender *</label>
            <div className="radio-group" style={{ marginTop: '0.6rem' }}>
              {['Male', 'Female', 'Other'].map(g => (
                <label key={g} className="radio-label">
                  <input
                    type="radio"
                    name="gender"
                    value={g}
                    checked={formData.gender === g}
                    onChange={handleChange}
                  />
                  {g}
                </label>
              ))}
            </div>
            {touched.gender && errors.gender && <p className="error-text">⚠ {errors.gender}</p>}
          </div>
        </div>

        {/* Skills / Interests (Checkboxes) */}
        <div className="form-group">
          <label className="form-label">Technical Skills & Interests *</label>
          <div className="checkbox-group">
            {AVAILABLE_SKILLS.map(skill => (
              <label key={skill} className="checkbox-label">
                <input
                  type="checkbox"
                  checked={formData.skills.includes(skill)}
                  onChange={() => handleSkillToggle(skill)}
                />
                {skill}
              </label>
            ))}
          </div>
```

```
          {touched.skills && errors.skills && <p className="error-text">⚠ {errors.skills}</p>}
        </div>

        {/* Address / Remarks */}
        <div className="form-group">
          <label className="form-label">Residential Address / City (Optional)</label>
          <textarea
            name="address"
            rows="2"
            placeholder="Enter student city or address details..."
            value={formData.address}
            onChange={handleChange}
            className="form-textarea"
          ></textarea>
        </div>

        {/* Form Actions */}
        <div className="form-actions">
          <button type="submit" className="btn btn-primary" style={{ flex: 1 }}>
            {editingStudent ? '✎ Update Student' : '➕ Register Student'}
          </button>
          {editingStudent && (
            <button type="button" className="btn btn-secondary" onClick={onCancelEdit}>
              Cancel
            </button>
          )}
        </div>
      </form>
    </div>
  );
}

export default StudentForm;
```

- **src/components/StudentTable.jsx :**

```
import React from 'react';

function StudentTable({ students, onEdit, onDelete }) {
  if (students.length === 0) {
    return (
      <div style={{ textAlign: 'center', padding: '3rem 1rem', color: 'var(--text-muted)' }}>
        <p style={{ fontSize: '2.5rem', marginBottom: '0.5rem' }}>📭</p>
        <h4>No Student Records Found</h4>
        <p style={{ fontSize: '0.88rem' }}>Register a student from the form on the left or clear
your filter.</p>
      </div>
    );
  }

  return (
    <div className="table-container">
      <table className="student-table">
        <thead>
          <tr>
            <th>Roll No</th>
            <th>Name & Email</th>
```

```
            <th>Dept & Sem</th>
            <th>Gender & Phone</th>
            <th>Skills</th>
            <th style={{ textAlign: 'center' }}>Actions</th>
          </tr>
        </thead>
        <tbody>
          {students.map((student) => (
            <tr key={student.id}>
              <td>
                <strong style={{ color: 'var(--primary-dark)' }}>{student.rollNo}</strong>
              </td>
              <td>
                <div style={{ fontWeight: 600 }}>{student.fullName}</div>
                <div style={{ fontSize: '0.78rem', color: 'var(--text-muted)'
}}>{student.email}</div>
              </td>
              <td>
                <div><span className="badge badge-dept">{student.department}</span></div>
                <div style={{ marginTop: '0.2rem' }}><span className="badge badge-
sem">{student.semester}</span></div>
              </td>
              <td>
                <div>{student.gender}</div>
                <div style={{ fontSize: '0.78rem', color: 'var(--text-muted)' }}>
{student.phone}</div>
              </td>
              <td>
                <div style={{ display: 'flex', flexWrap: 'wrap', gap: '0.2rem', maxWidth: '200px'
}}>
                  {student.skills && student.skills.map(s => (
                    <span key={s} className="badge badge-skill">{s}</span>
                  ))}
                </div>
              </td>
              <td>
                <div style={{ display: 'flex', gap: '0.4rem', justifyContent: 'center' }}>
                  <button
                    className="btn btn-outline-primary btn-sm"
                    onClick={() => onEdit(student)}
                    title="Edit Record"
                  >
                    
                  </button>
                  <button
                    className="btn btn-outline-danger btn-sm"
                    onClick={() => onDelete(student.id, student.fullName)}
                    title="Delete Record"
                  >
                    
                  </button>
                </div>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
```

```
export default StudentTable;
```

- **src/components/StatsOverview.jsx :**

```
import React from 'react';

function StatsOverview({ students }) {
  const totalStudents = students.length;
  const cseCount = students.filter(s => s.department === 'Computer Engineering').length;
  const itCount = students.filter(s => s.department === 'Information Technology').length;
  const aiCount = students.filter(s => s.department === 'AI & Data Science').length;

  return (
    <div className="stats-grid">
      <div className="stat-card">
        <div className="stat-icon" style={{ background: '#ecfdf5', color: '#059669' }}>□</div>
        <div>
          <div className="stat-number">{totalStudents}</div>
          <div className="stat-label">Total Enrolled Students</div>
        </div>
      </div>

      <div className="stat-card">
        <div className="stat-icon" style={{ background: '#eff6ff', color: '#2563eb' }}>□</div>
        <div>
          <div className="stat-number">{cseCount}</div>
          <div className="stat-label">Computer Engg</div>
        </div>
      </div>

      <div className="stat-card">
        <div className="stat-icon" style={{ background: '#f5f3ff', color: '#7c3aed' }}>□</div>
        <div>
          <div className="stat-number">{itCount}</div>
          <div className="stat-label">Information Tech</div>
        </div>
      </div>

      <div className="stat-card">
        <div className="stat-icon" style={{ background: '#fffbeb', color: '#d97706' }}>□</div>
        <div>
          <div className="stat-number">{aiCount}</div>
          <div className="stat-label">AI & Data Science</div>
        </div>
      </div>
    </div>
  );
}

export default StatsOverview;
```

- **src/utils/validation.js :**

```
/**
 * Utility for Form Validation rules
 */
export const validateStudentForm = (formData, existingStudents = [], currentEditingId = null) =>
{
  const errors = {};

  // 1. Full Name Validation
  if (!formData.fullName || !formData.fullName.trim()) {
    errors.fullName = 'Full Name is required.';
  } else if (formData.fullName.trim().length < 3) {
    errors.fullName = 'Full Name must be at least 3 characters.';
  } else if (!/^[a-zA-Z\s]+$/.test(formData.fullName.trim())) {
    errors.fullName = 'Full Name can only contain letters and spaces.';
  }

  // 2. Roll Number Validation
  if (!formData.rollNo || !formData.rollNo.trim()) {
    errors.rollNo = 'Enrollment / Roll No is required.';
  } else if (!/^[A-Za-z0-9_-]{3,15}$/.test(formData.rollNo.trim())) {
    errors.rollNo = 'Roll No must be 3-15 alphanumeric characters.';
  } else {
    // Check uniqueness
    const duplicate = existingStudents.find(
      s => s.rollNo.toLowerCase() === formData.rollNo.trim().toLowerCase() && s.id !==
currentEditingId
    );
    if (duplicate) {
      errors.rollNo = 'This Roll No is already registered.';
    }
  }

  // 3. Email Validation
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!formData.email || !formData.email.trim()) {
    errors.email = 'Email address is required.';
  } else if (!emailRegex.test(formData.email.trim())) {
    errors.email = 'Please enter a valid email address (e.g. name@domain.com).';
  }

  // 4. Phone Number Validation
  const phoneRegex = /^[6-9]\d{9}$/;
  if (!formData.phone || !formData.phone.trim()) {
    errors.phone = '10-digit Mobile Number is required.';
  } else if (!phoneRegex.test(formData.phone.trim())) {
    errors.phone = 'Enter a valid 10-digit Indian mobile number starting with 6-9.';
  }

  // 5. Department / Course Selection
  if (!formData.department) {
    errors.department = 'Please select a department / branch.';
  }

  // 6. Semester Selection
  if (!formData.semester) {
    errors.semester = 'Please select the current semester.';
  }

  // 7. Gender Validation
  if (!formData.gender) {
```

```
    errors.gender = 'Please select gender.';
  }

  // 8. Skills / Interests Validation (Minimum 1 skill selected)
  if (!formData.skills || formData.skills.length === 0) {
    errors.skills = 'Please select at least one skill or interest.';
  }

  return {
    isValid: Object.keys(errors).length === 0,
    errors
  };
};
```

- **src/index.css :**

```
:root {
  --primary: #059669;
  --primary-dark: #047857;
  --primary-light: #d1fae5;
  --accent: #3b82f6;
  --danger: #ef4444;
  --danger-light: #fee2e2;
  --warning: #f59e0b;
  --bg-main: #f8fafc;
  --bg-card: #ffffff;
  --text-main: #0f172a;
  --text-muted: #64748b;
  --border-color: #e2e8f0;
  --radius: 12px;
  --font-family: 'Inter', system-ui, sans-serif;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: var(--font-family);
  background-color: var(--bg-main);
  color: var(--text-main);
  line-height: 1.6;
}

.app-container {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

.main-content {
  flex: 1;
  max-width: 1300px;
  margin: 0 auto;
  padding: 2rem 1.5rem;
```

```
  width: 100%;
}

.header {
  background: linear-gradient(135deg, #064e3b 0%, #065f46 50%, #059669 100%);
  color: white;
  padding: 2.5rem 1.5rem;
  text-align: center;
  box-shadow: 0 4px 6px -1px rgb(0 0 0 / 0.1);
}

.header-badge {
  display: inline-block;
  background: rgba(255, 255, 255, 0.2);
  padding: 0.35rem 1rem;
  border-radius: 9999px;
  font-size: 0.85rem;
  font-weight: 700;
  text-transform: uppercase;
  margin-bottom: 0.75rem;
}

.header-title {
  font-size: 2.25rem;
  font-weight: 800;
  margin-bottom: 0.5rem;
}

.header-subtitle {
  font-size: 1.05rem;
  color: #a7f3d0;
  max-width: 700px;
  margin: 0 auto;
}

/* Stats Cards */
.stats-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
  margin-bottom: 2rem;
}

.stat-card {
  background: white;
  border: 1px solid var(--border-color);
  border-radius: var(--radius);
  padding: 1.25rem;
  display: flex;
  align-items: center;
  gap: 1rem;
  box-shadow: 0 1px 3px rgba(0,0,0,0.05);
}

.stat-icon {
  width: 48px;
  height: 48px;
  border-radius: 10px;
  display: flex;
  align-items: center;
```

```
  justify-content: center;
  font-size: 1.5rem;
}

.stat-number {
  font-size: 1.6rem;
  font-weight: 800;
  color: var(--text-main);
  line-height: 1.1;
}

.stat-label {
  font-size: 0.85rem;
  color: var(--text-muted);
}

/* Main Two Columns */
.layout-grid {
  display: grid;
  grid-template-columns: 420px 1fr;
  gap: 2rem;
  align-items: start;
}

@media (max-width: 950px) {
  .layout-grid {
    grid-template-columns: 1fr;
  }
}

.card {
  background: white;
  border: 1px solid var(--border-color);
  border-radius: var(--radius);
  padding: 1.75rem;
  box-shadow: 0 2px 4px rgba(0,0,0,0.04);
}

.card-title {
  font-size: 1.25rem;
  font-weight: 700;
  margin-bottom: 1.25rem;
  display: flex;
  align-items: center;
  justify-content: space-between;
}

/* Form Styles */
.form-group {
  margin-bottom: 1rem;
}

.form-label {
  display: block;
  font-size: 0.85rem;
  font-weight: 600;
  color: var(--text-main);
  margin-bottom: 0.3rem;
}
```

```css
.form-input, .form-select, .form-textarea {
  width: 100%;
  padding: 0.65rem 0.85rem;
  font-size: 0.9rem;
  border: 1px solid var(--border-color);
  border-radius: 8px;
  outline: none;
  font-family: inherit;
  transition: all 0.2s;
}

.form-input:focus, .form-select:focus, .form-textarea:focus {
  border-color: var(--primary);
  box-shadow: 0 0 0 3px rgba(5, 150, 105, 0.15);
}

.form-input.has-error, .form-select.has-error {
  border-color: var(--danger);
  background-color: #fffafb;
}

.error-text {
  color: var(--danger);
  font-size: 0.78rem;
  margin-top: 0.25rem;
  display: flex;
  align-items: center;
  gap: 0.25rem;
}

.radio-group, .checkbox-group {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
  margin-top: 0.35rem;
}

.radio-label, .checkbox-label {
  display: inline-flex;
  align-items: center;
  gap: 0.4rem;
  font-size: 0.88rem;
  cursor: pointer;
}

.form-actions {
  display: flex;
  gap: 0.75rem;
  margin-top: 1.5rem;
}

/* Buttons */
.btn {
  padding: 0.6rem 1.2rem;
  font-size: 0.88rem;
  font-weight: 600;
  border-radius: 8px;
  border: none;
  cursor: pointer;
  display: inline-flex;
```

```
  align-items: center;
  gap: 0.4rem;
  transition: all 0.2s;
  font-family: inherit;
}

.btn-primary { background: var(--primary); color: white; }
.btn-primary:hover { background: var(--primary-dark); }
.btn-secondary { background: #e2e8f0; color: #334155; }
.btn-secondary:hover { background: #cbd5e1; }
.btn-danger { background: var(--danger); color: white; }
.btn-danger:hover { background: #dc2626; }
.btn-outline-danger { background: transparent; border: 1px solid var(--danger); color: var(--
danger); }
.btn-outline-danger:hover { background: var(--danger-light); }
.btn-outline-primary { background: transparent; border: 1px solid var(--primary); color: var(--
primary); }
.btn-outline-primary:hover { background: var(--primary-light); }
.btn-sm { padding: 0.35rem 0.65rem; font-size: 0.8rem; }

/* Table Styles */
.table-container {
  overflow-x: auto;
}

.student-table {
  width: 100%;
  border-collapse: collapse;
  text-align: left;
  font-size: 0.88rem;
}

.student-table th {
  background: #f1f5f9;
  padding: 0.75rem 1rem;
  font-weight: 700;
  color: var(--text-main);
  border-bottom: 2px solid var(--border-color);
}

.student-table td {
  padding: 0.85rem 1rem;
  border-bottom: 1px solid var(--border-color);
  vertical-align: middle;
}

.student-table tr:hover {
  background: #f8fafc;
}

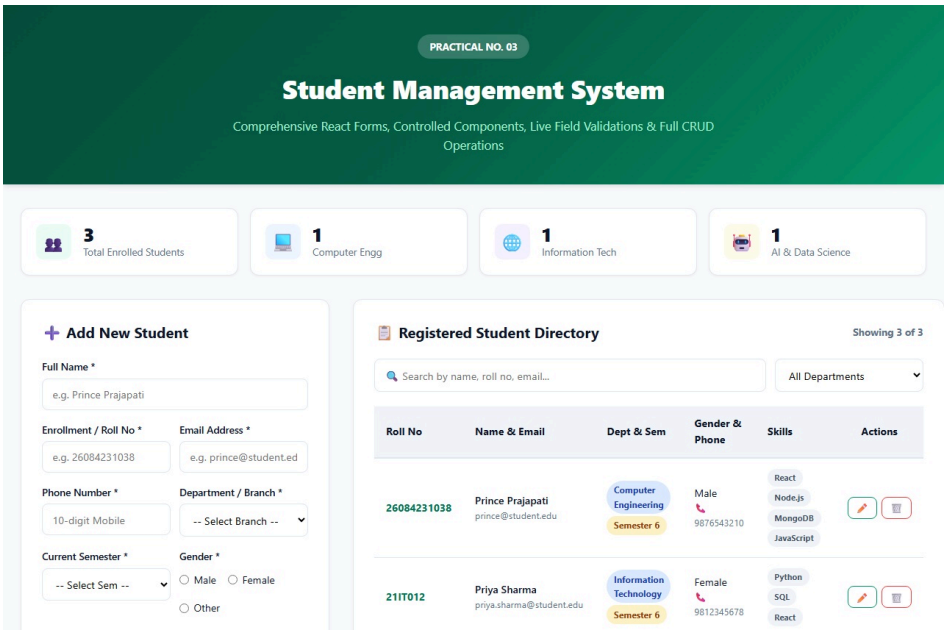
.badge {
  display: inline-block;
  padding: 0.2rem 0.6rem;
  border-radius: 9999px;
  font-size: 0.75rem;
  font-weight: 700;
}

.badge-dept { background: #dbeafe; color: #1e40af; }
.badge-sem { background: #fef3c7; color: #92400e; }
```

```css
.badge-skill { background: #f1f5f9; color: #475569; margin-right: 0.25rem; font-size: 0.7rem; }

.footer {
  background: #0f172a;
  color: #94a3b8;
  padding: 2rem 1.5rem;
  text-align: center;
  font-size: 0.9rem;
  border-top: 1px solid #1e293b;
}
```

4. Output :



Practical – 04 : Build a Single Page Application (SPA) using React Router DOM with Dynamic Routing, Nested Navigation, and URL Parameters.

1. Commands :

```
npm create vite@latest MERN_04
cd MERN_04
npm install react-router-dom
npm run dev
```

2. Project structure :

```
MERN_04/
|-- src/
|   |-- components/
|   |   |-- Navbar.jsx
|   |   |-- Footer.jsx
|   |-- pages/
|   |   |-- Home.jsx
|   |   |-- About.jsx
|   |   |-- Courses.jsx
|   |   |-- CourseDetails.jsx
|   |   |-- Contact.jsx
|   |   |-- NotFound.jsx
|   |-- data/
|   |   |-- coursesData.js
|   |-- App.jsx
|   |-- index.css
|   |-- main.jsx
|-- package.json
|-- index.html
```

3. Source Code :

- **src/App.jsx :**

```jsx
import React from 'react';
import { Routes, Route } from 'react-router-dom';
import Navbar from './components/Navbar';
import Footer from './components/Footer';
import Home from './pages/Home';
import About from './pages/About';
import Courses from './pages/Courses';
import CourseDetails from './pages/CourseDetails';
import Contact from './pages/Contact';
import NotFound from './pages/NotFound';

function App() {
  return (
    <div className="app-wrapper">
      {/* Persistent Navigation Bar */}
      <Navbar />

      {/* Main Content Area Rendering Current Route */}
      <main className="main-content">
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/about" element={<About />} />
          <Route path="/courses" element={<Courses />} />
          <Route path="/courses/:courseId" element={<CourseDetails />} />
```

```
          <Route path="/contact" element={<Contact />} />
          {/* Wildcard 404 Route */}
          <Route path="*" element={<NotFound />} />
        </Routes>
      </main>

      {/* Persistent Footer */}
      <Footer />
    </div>
  );
}

export default App;
```

- **src/components/Navbar.jsx :**

```
import React from 'react';
import { NavLink, Link } from 'react-router-dom';

function Navbar() {
  return (
    <nav className="navbar">
      <Link to="/" className="nav-brand">
        <span>⚡</span> EduTech SPA
      </Link>

      <ul className="nav-links">
        <li className="nav-item">
          <NavLink to="/" end>Home</NavLink>
        </li>
        <li className="nav-item">
          <NavLink to="/about">About Us</NavLink>
        </li>
        <li className="nav-item">
          <NavLink to="/courses">Courses</NavLink>
        </li>
        <li className="nav-item">
          <NavLink to="/contact">Contact</NavLink>
        </li>
      </ul>
    </nav>
  );
}

export default Navbar;
```

- **src/pages/Home.jsx :**

```
import React from 'react';
import { Link } from 'react-router-dom';
import { COURSES } from '../data/coursesData';

function Home() {
  const featuredCourses = COURSES.slice(0, 3);
```

```
  return (
    <div>
      {/* Hero Section */}
      <section className="hero-section">
        <span className="badge badge-purple" style={{ marginBottom: '1rem' }}>
          Practical 04 Demonstration
        </span>
        <h1 className="hero-title">Single Page Application with React Router</h1>
        <p className="hero-subtitle">
          Experience instant client-side transitions, deep dynamic linking, active navigation
styling,
          and breadcrumb path tracking without full-page reloads.
        </p>
        <div className="hero-buttons">
          <Link to="/courses" className="btn btn-primary">
            Explore All Courses →
          </Link>
          <Link to="/about" className="btn btn-outline">
            About This Practical
          </Link>
        </div>
      </section>

      {/* Highlights Grid */}
      <section style={{ marginBottom: '3rem' }}>
        <h2 style={{ fontSize: '1.6rem', fontWeight: 800, marginBottom: '1.5rem', textAlign:
'center' }}>
          ✨React Router Architecture Highlights
        </h2>

        <div className="grid-3">
          <div className="card">
            <div style={{ fontSize: '2rem', marginBottom: '0.75rem' }}>□</div>
            <h3 style={{ fontSize: '1.2rem', fontWeight: 700, marginBottom: '0.5rem' }}>Client-
Side Routing</h3>
            <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}>
              Transitions between views happen dynamically in browser memory without sending full
document requests to the server.
            </p>
          </div>

          <div className="card">
            <div style={{ fontSize: '2rem', marginBottom: '0.75rem' }}>□</div>
            <h3 style={{ fontSize: '1.2rem', fontWeight: 700, marginBottom: '0.5rem' }}>Dynamic
URL Parameters</h3>
            <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}>
              Routes like <code>/courses/:courseId</code> extract URL slug variables via the
<code>useParams()</code> hook.
            </p>
          </div>

          <div className="card">
            <div style={{ fontSize: '2rem', marginBottom: '0.75rem' }}>□</div>
            <h3 style={{ fontSize: '1.2rem', fontWeight: 700, marginBottom: '0.5rem' }}>Catch-All
404 Pages</h3>
            <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}>
              Unmatched route paths are automatically caught by the wildcard
<code>path="*"</code> fallback handler.
            </p>
```

```
          </div>
        </div>
      </section>

      {/* Featured Courses */}
      <section>
        <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center',
marginBottom: '1.5rem' }}>
          <h2 style={{ fontSize: '1.5rem', fontWeight: 800 }}>Featured Specializations</h2>
          <Link to="/courses" style={{ color: 'var(--primary)', textDecoration: 'none',
fontWeight: 600, fontSize: '0.95rem' }}>
            View All Courses &rarr;
          </Link>
        </div>

        <div className="grid-3">
          {featuredCourses.map(course => (
            <div key={course.id} className="card">
              <div style={{ display: 'flex', justifyContent: 'space-between', marginBottom:
'0.75rem' }}>
                <span className="badge badge-purple">{course.category}</span>
                <span style={{ fontSize: '0.85rem', fontWeight: 600, color: 'var(--text-muted)'
}}>★ {course.rating}</span>
              </div>
              <h3 style={{ fontSize: '1.2rem', fontWeight: 700, marginBottom: '0.5rem'
}}>{course.title}</h3>
              <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem', marginBottom:
'1.25rem', flex: 1 }}>
                {course.shortDesc}
              </p>
              <div style={{ display: 'flex', justifyContent: 'space-between', alignItems:
'center', borderTop: '1px solid var(--border-color)', paddingTop: '1rem' }}>
                <span style={{ fontSize: '0.85rem', color: 'var(--text-muted)' }}>
{course.duration}</span>
                <Link to={`/courses/${course.id}`} className="btn btn-primary btn-sm">
                  View Details &rarr;
                </Link>
              </div>
            </div>
          ))}
        </div>
      </section>
    </div>
  );
}

export default Home;
```

- **src/pages/Courses.jsx :**

```
import React, { useState } from 'react';
import { Link } from 'react-router-dom';
import { COURSES } from '../data/coursesData';

function Courses() {
  const [selectedCategory, setSelectedCategory] = useState('All');
  const [search, setSearch] = useState('');
```

```
  const categories = ['All', 'Frontend', 'Backend', 'Full Stack', 'Database'];

  const filtered = COURSES.filter(course => {
    const matchCategory = selectedCategory === 'All' || course.category === selectedCategory;
    const matchSearch = course.title.toLowerCase().includes(search.toLowerCase()) ||
                        course.shortDesc.toLowerCase().includes(search.toLowerCase());
    return matchCategory && matchSearch;
  });

  return (
    <div>
      <div className="breadcrumb">
        <Link to="/">Home</Link> <span>/</span> <span>Courses Catalog</span>
      </div>

      <div style={{ marginBottom: '2rem' }}>
        <h1 style={{ fontSize: '2.25rem', fontWeight: 800, marginBottom: '0.5rem' }}>
          Explore Technical Specializations
        </h1>
        <p style={{ color: 'var(--text-muted)', fontSize: '1rem' }}>
          Click on any course card to test <strong>Dynamic URL Route Parameters</strong> with
<code>/courses/:courseId</code>.
        </p>
      </div>

      {/* Filter and Search Bar */}
      <div style={{ display: 'flex', gap: '1rem', flexWrap: 'wrap', marginBottom: '2rem',
alignItems: 'center' }}>
        <input
          type="text"
          placeholder="🔍 Search courses by keyword..."
          value={search}
          onChange={(e) => setSearch(e.target.value)}
          style={{
            flex: 1,
            minWidth: '250px',
            padding: '0.65rem 1rem',
            borderRadius: '8px',
            border: '1px solid var(--border-color)',
            outline: 'none',
            fontSize: '0.95rem'
          }}
        />

        <div style={{ display: 'flex', gap: '0.4rem', flexWrap: 'wrap' }}>
          {categories.map(cat => (
            <button
              key={cat}
              onClick={() => setSelectedCategory(cat)}
              className={`btn btn-sm ${selectedCategory === cat ? 'btn-primary' : 'btn-
outline'}`}
            >
              {cat}
            </button>
          ))}
        </div>
      </div>

      {/* Courses Grid */}
```

```
      {filtered.length === 0 ? (
        <div style={{ textAlign: 'center', padding: '3rem', color: 'var(--text-muted)' }}>
          <h3>No courses matched your query.</h3>
        </div>
      ) : (
        <div className="grid-2">
          {filtered.map(course => (
            <div key={course.id} className="card">
              <div style={{ display: 'flex', justifyContent: 'space-between', alignItems:
'center', marginBottom: '0.75rem' }}>
                <span className="badge badge-purple">{course.category}</span>
                <span style={{ fontSize: '0.85rem', fontWeight: 600, color: 'var(--text-muted)'
}}>
                  □ {course.level}
                </span>
              </div>

              <h2 style={{ fontSize: '1.35rem', fontWeight: 700, marginBottom: '0.5rem' }}>
                {course.title}
              </h2>
              <p style={{ color: 'var(--text-muted)', fontSize: '0.925rem', marginBottom:
'1.25rem', flex: 1 }}>
                {course.shortDesc}
              </p>

              <div style={{ fontSize: '0.85rem', color: 'var(--text-muted)', marginBottom: '1rem'
}}>
                <p>□□ <strong>Instructor:</strong> {course.instructor}</p>
                <p>□ <strong>Duration:</strong> {course.duration} &bull; ★ {course.rating} /
5.0</p>
              </div>

              <div style={{ borderTop: '1px solid var(--border-color)', paddingTop: '1rem',
display: 'flex', justifyContent: 'space-between', alignItems: 'center' }}>
                <span style={{ fontSize: '0.85rem', color: 'var(--text-muted)' }}>
                  □ {course.enrolledStudents} learners
                </span>
                <Link to={`/courses/${course.id}`} className="btn btn-primary btn-sm">
                  View Syllabus & Details →
                </Link>
              </div>
            </div>
          ))}
        </div>
      )}
    </div>
  );
}

export default Courses;
```

- **src/pages/CourseDetails.jsx :**

```
import React from 'react';
import { useParams, Link, useNavigate } from 'react-router-dom';
import { COURSES } from '../data/coursesData';
```

```
function CourseDetails() {
  const { courseId } = useParams();
  const navigate = useNavigate();

  const course = COURSES.find(c => c.id === courseId);

  if (!course) {
    return (
      <div style={{ textAlign: 'center', padding: '4rem 1rem' }}>
        <h2 style={{ fontSize: '2rem', marginBottom: '1rem', color: '#ef4444' }}>Course Not
Found</h2>
        <p style={{ color: 'var(--text-muted)', marginBottom: '1.5rem' }}>
          No course found with URL slug: <code>{courseId}</code>
        </p>
        <Link to="/courses" className="btn btn-primary">
          ← Back to Courses
        </Link>
      </div>
    );
  }

  return (
    <div>
      {/* Breadcrumb Navigation */}
      <div className="breadcrumb">
        <Link to="/">Home</Link> <span>/</span> <Link to="/courses">Courses</Link> <span>/</span>
<span>{course.title}</span>
      </div>

      <div style={{ maxWidth: '900px', margin: '0 auto' }}>
        {/* Course Header Banner */}
        <div className="card" style={{ background: 'linear-gradient(135deg, #1e1b4b 0%, #312e81
100%)', color: 'white', marginBottom: '2rem' }}>
          <div style={{ display: 'flex', gap: '0.5rem', marginBottom: '1rem' }}>
            <span className="badge badge-purple" style={{ background: 'rgba(255,255,255,0.2)',
color: 'white' }}>
              {course.category}
            </span>
            <span className="badge badge-amber">
              {course.level}
            </span>
          </div>

          <h1 style={{ fontSize: '2.4rem', fontWeight: 800, marginBottom: '0.75rem', lineHeight:
1.2 }}>
            {course.title}
          </h1>

          <p style={{ fontSize: '1.1rem', color: '#c7d2fe', marginBottom: '1.5rem', lineHeight:
1.6 }}>
            {course.fullDesc}
          </p>

          <div style={{ display: 'flex', gap: '2rem', flexWrap: 'wrap', fontSize: '0.95rem',
borderTop: '1px solid rgba(255,255,255,0.15)', paddingTop: '1.25rem' }}>
            <div>👨‍🏫 <strong>Instructor:</strong> {course.instructor}</div>
            <div>⏱ <strong>Duration:</strong> {course.duration}</div>
            <div>★ <strong>Rating:</strong> {course.rating} / 5.0</div>
            <div>👥 <strong>Enrolled:</strong> {course.enrolledStudents} students</div>
          </div>
```

```
        </div>

        {/* Dynamic Route Info Box */}
        <div style={{
          background: '#f0fdf4',
          border: '1px solid #bbf7d0',
          borderRadius: '12px',
          padding: '1.25rem',
          marginBottom: '2rem',
          fontSize: '0.9rem',
          color: '#166534'
        }}>
          □ <strong>Dynamic Parameter Inspection:</strong> This page received URL parameter
<code>useParams().courseId = "{courseId}"</code>.
        </div>

        {/* Course Modules */}
        <div className="card" style={{ marginBottom: '2rem' }}>
          <h3 style={{ fontSize: '1.3rem', fontWeight: 700, marginBottom: '1rem' }}>□ Course
Curriculum & Modules</h3>
          <ul style={{ listStyle: 'none', display: 'flex', flexDirection: 'column', gap:
'0.75rem' }}>
            {course.modules.map((mod, index) => (
              <li key={index} style={{
                display: 'flex',
                alignItems: 'center',
                gap: '0.75rem',
                padding: '0.75rem 1rem',
                background: '#f8fafc',
                borderRadius: '8px',
                border: '1px solid var(--border-color)',
                fontSize: '0.95rem'
              }}>
                <span style={{
                  background: 'var(--primary-light)',
                  color: 'var(--primary)',
                  fontWeight: 700,
                  width: '28px',
                  height: '28px',
                  borderRadius: '50%',
                  display: 'flex',
                  alignItems: 'center',
                  justifyContent: 'center',
                  fontSize: '0.85rem'
                }}>
                  {index + 1}
                </span>
                <span>{mod}</span>
              </li>
            ))}
          </ul>
        </div>

        {/* Prerequisites & Action */}
        <div className="card" style={{ marginBottom: '2rem' }}>
          <h3 style={{ fontSize: '1.3rem', fontWeight: 700, marginBottom: '0.75rem'
}}>Prerequisites</h3>
          <div style={{ display: 'flex', gap: '0.5rem', flexWrap: 'wrap', marginBottom: '1.5rem'
}}>
            {course.prerequisites.map(p => (
```

```
              <span key={p} className="badge badge-blue">{p}</span>
            ))}
          </div>

          <div style={{ display: 'flex', gap: '1rem', borderTop: '1px solid var(--border-color)',
paddingTop: '1.25rem' }}>
            <button
              className="btn btn-primary"
              onClick={() => alert(`  You have registered for: ${course.title}`)}
            >
               Enroll in this Specialization
            </button>
            <button
              className="btn btn-outline"
              onClick={() => navigate('/courses')}
            >
              ← Back to Catalog (useNavigate)
            </button>
          </div>
        </div>
      </div>
    </div>
  );
}

export default CourseDetails;
```

- **src/pages/About.jsx :**

```
import React from 'react';
import { Link } from 'react-router-dom';

function About() {
  return (
    <div>
      <div className="breadcrumb">
        <Link to="/">Home</Link> <span>/</span> <span>About Us</span>
      </div>

      <div style={{ maxWidth: '850px', margin: '0 auto' }}>
        <h1 style={{ fontSize: '2.25rem', fontWeight: 800, marginBottom: '1rem', color: '#1e1b4b'
}}>
          About Our Single Page Application
        </h1>
        <p style={{ fontSize: '1.1rem', color: 'var(--text-muted)', lineHeight: 1.7,
marginBottom: '2rem' }}>
          This project illustrates the core mechanics of modern client-side routing in single-
page applications.
          By utilizing the HTML5 History API (via React Router), our application provides an
instant, desktop-app like
          user experience with instantaneous page switches.
        </p>

        <div className="card" style={{ marginBottom: '2rem' }}>
          <h3 style={{ fontSize: '1.25rem', fontWeight: 700, marginBottom: '1rem' }}>
             Core Concepts Covered in Practical 04:
          </h3>
```

```
          <ul style={{ paddingLeft: '1.5rem', lineHeight: 2, color: 'var(--text-main)', fontSize:
'0.95rem' }}>
            <li><strong>BrowserRouter:</strong> Wraps the root application and syncs the UI with
the browser address bar.</li>
            <li><strong>Routes & Route:</strong> Declaratively maps URL path patterns to specific
page components.</li>
            <li><strong>Link & NavLink:</strong> Custom navigation components that intercept
browser clicks and avoid page refreshes.</li>
            <li><strong>useParams Hook:</strong> Reads dynamic URL parameters (e.g.,
<code>:courseId</code>) directly into components.</li>
            <li><strong>useNavigate Hook:</strong> Programmatically triggers navigation from
handlers and button callbacks.</li>
            <li><strong>Wildcard Route (`*`):</strong> Seamlessly handles 404 errors when
visitors input unknown URLs.</li>
          </ul>
        </div>

        <div className="grid-2">
          <div className="card">
            <h4 style={{ fontSize: '1.1rem', fontWeight: 700, marginBottom: '0.5rem' }}>⚡ Zero
Page Reloads</h4>
            <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}>
              All navigation happens without white flashes or waiting for the server to render
full HTML documents.
            </p>
          </div>

          <div className="card">
            <h4 style={{ fontSize: '1.1rem', fontWeight: 700, marginBottom: '0.5rem' }}>🔗
Bookmarkable URLs</h4>
            <p style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}>
              Every view has a unique address bar URL that can be directly bookmarked, refreshed,
or shared.
            </p>
          </div>
        </div>
      </div>
    </div>
  );
}

export default About;
```

- **src/pages/Contact.jsx :**

```
import React, { useState } from 'react';
import { Link } from 'react-router-dom';

function Contact() {
  const [submitted, setSubmitted] = useState(false);
  const [formData, setFormData] = useState({ name: '', email: '', subject: '', message: '' });

  const handleSubmit = (e) => {
    e.preventDefault();
    setSubmitted(true);
  };
```

```
  return (
    <div>
      <div className="breadcrumb">
        <Link to="/">Home</Link> <span>/</span> <span>Contact</span>
      </div>

      <div style={{ maxWidth: '850px', margin: '0 auto' }}>
        <h1 style={{ fontSize: '2.25rem', fontWeight: 800, marginBottom: '0.5rem' }}>Contact
Department Support</h1>
        <p style={{ color: 'var(--text-muted)', marginBottom: '2rem' }}>
          Have questions regarding the curriculum or practical journal? Send us a message.
        </p>

        <div className="grid-2">
          {/* Contact Details Card */}
          <div className="card" style={{ background: '#f8fafc' }}>
            <h3 style={{ fontSize: '1.25rem', fontWeight: 700, marginBottom: '1rem' }}>
Department Office</h3>
            <div style={{ display: 'flex', flexDirection: 'column', gap: '1rem', fontSize:
'0.925rem', color: 'var(--text-muted)' }}>
              <p> <strong>Address:</strong> Department of Computer Engineering, Technology
Campus</p>
              <p> <strong>Email:</strong> webtech.lab@college.edu</p>
              <p> <strong>Phone:</strong> +91 98765 43210</p>
              <p> <strong>Lab Hours:</strong> Mon - Fri: 9:00 AM – 5:00 PM</p>
            </div>
          </div>

          {/* Contact Form */}
          <div className="card">
            {submitted ? (
              <div style={{ textAlign: 'center', padding: '2rem 1rem' }}>
                <div style={{ fontSize: '3rem', marginBottom: '0.5rem' }}></div>
                <h3 style={{ fontSize: '1.25rem', fontWeight: 700, color: 'var(--primary)'
}}>Message Received!</h3>
                <p style={{ color: 'var(--text-muted)', margin: '0.75rem 0 1.5rem', fontSize:
'0.9rem' }}>
                  Thank you, <strong>{formData.name}</strong>. We will get back to your email
({formData.email}) shortly.
                </p>
                <button
                  className="btn btn-outline btn-sm"
                  onClick={() => { setSubmitted(false); setFormData({ name: '', email: '',
subject: '', message: '' }); }}
                >
                  Send Another Inquiry
                </button>
              </div>
            ) : (
              <form onSubmit={handleSubmit}>
                <div style={{ marginBottom: '1rem' }}>
                  <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: 600,
marginBottom: '0.25rem' }}>Full Name</label>
                  <input
                    type="text"
                    required
                    placeholder="Your Name"
                    value={formData.name}
                    onChange={(e) => setFormData({ ...formData, name: e.target.value })}
                    style={{ width: '100%', padding: '0.6rem 0.8rem', borderRadius: '8px',
```

```
border: '1px solid var(--border-color)', outline: 'none' }}
                  />
                </div>

                <div style={{ marginBottom: '1rem' }}>
                  <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: 600,
marginBottom: '0.25rem' }}>Email Address</label>
                  <input
                    type="email"
                    required
                    placeholder="Your Email"
                    value={formData.email}
                    onChange={(e) => setFormData({ ...formData, email: e.target.value })}
                    style={{ width: '100%', padding: '0.6rem 0.8rem', borderRadius: '8px',
border: '1px solid var(--border-color)', outline: 'none' }}
                  />
                </div>

                <div style={{ marginBottom: '1rem' }}>
                  <label style={{ display: 'block', fontSize: '0.85rem', fontWeight: 600,
marginBottom: '0.25rem' }}>Message</label>
                  <textarea
                    rows="3"
                    required
                    placeholder="Your query or feedback..."
                    value={formData.message}
                    onChange={(e) => setFormData({ ...formData, message: e.target.value })}
                    style={{ width: '100%', padding: '0.6rem 0.8rem', borderRadius: '8px',
border: '1px solid var(--border-color)', outline: 'none', fontFamily: 'inherit' }}
                  ></textarea>
                </div>

                <button type="submit" className="btn btn-primary" style={{ width: '100%' }}>
                  Send Message
                </button>
              </form>
            )}
          </div>
        </div>
      </div>
    </div>
  );
}

export default Contact;
```

- **src/data/coursesData.js :**

```
export const COURSES = [
  {
    id: 'react-mastery',
    title: 'Modern React.js & Redux Toolkit',
    shortDesc: 'Master Functional Components, React Hooks, Context API, and modern state
architecture.',
    fullDesc: 'This comprehensive course takes you from fundamental React concepts to advanced
enterprise patterns. You will build single page applications using React Router, manage complex
global states with Redux Toolkit, and master custom hooks for optimal performance.',
```

```
    duration: '8 Weeks',
    level: 'Intermediate',
    category: 'Frontend',
    instructor: 'Dr. Sarah Jenkins',
    rating: 4.9,
    enrolledStudents: 1240,
    modules: [
      'React Fundamentals & JSX Syntax',
      'Hooks in Depth (useState, useEffect, useMemo, useCallback)',
      'Single Page Routing with React Router v6',
      'Global State Management & Performance Optimization',
      'Testing with Vitest and React Testing Library'
    ],
    prerequisites: ['Basic JavaScript (ES6+)', 'HTML5 & CSS3']
  },
  {
    id: 'nodejs-express-api',
    title: 'Backend Engineering with Node.js & Express',
    shortDesc: 'Build secure, scalable RESTful APIs with Express, JWT authentication, and
middleware.',
    fullDesc: 'Learn how to architect robust backend web servers using Node.js and Express.js
framework. Discover middleware design patterns, error handling, rate limiting, REST best
practices, and integration with databases.',
    duration: '6 Weeks',
    level: 'Intermediate',
    category: 'Backend',
    instructor: 'Prof. Michael Chang',
    rating: 4.8,
    enrolledStudents: 980,
    modules: [
      'Node.js Event Loop & Asynchronous Architecture',
      'Express.js Routing, Middleware & Controllers',
      'REST API Design & HTTP Status Codes',
      'Authentication & Authorization with JWT',
      'API Security, Rate Limiting & Deployment'
    ],
    prerequisites: ['Core JavaScript', 'Understanding of HTTP Protocol']
  },
  {
    id: 'fullstack-mern',
    title: 'Full Stack MERN Developer Bootcamp',
    shortDesc: 'End-to-end web applications combining MongoDB, Express, React, and Node.js.',
    fullDesc: 'Transform into a full-stack engineer by connecting MongoDB databases, Express
backends, and responsive React user interfaces into production-ready web platforms.',
    duration: '12 Weeks',
    level: 'Advanced',
    category: 'Full Stack',
    instructor: 'Alex Rivera',
    rating: 4.95,
    enrolledStudents: 2150,
    modules: [
      'Full Stack Architecture & Monorepos',
      'Mongoose Schemas, Models & MongoDB Atlas',
      'CRUD API Integration with React Query / Axios',
      'Stateful Authentication & Protected Routes',
      'Full CI/CD Deployment to Cloud Services'
    ],
    prerequisites: ['React Basics', 'Node.js Basics']
  },
  {
```

```
    id: 'mongodb-database-design',
    title: 'MongoDB Atlas & NoSQL Architecture',
    shortDesc: 'Data modeling, indexing, aggregation pipelines, and cloud database
optimization.',
    fullDesc: 'Deep dive into document-oriented database design with MongoDB. Learn schema
optimization, indexes for lightning fast queries, complex aggregation pipelines, and cloud
clustering with MongoDB Atlas.',
    duration: '4 Weeks',
    level: 'Beginner',
    category: 'Database',
    instructor: 'Pooja Patel',
    rating: 4.7,
    enrolledStudents: 650,
    modules: [
      'Document DB vs Relational Databases',
      'MongoDB Atlas Setup & Security Configuration',
      'CRUD Operations & Query Operators',
      'Aggregation Pipeline Mastery',
      'Indexing & Query Performance Tuning'
    ],
    prerequisites: ['Basic understanding of JSON & databases']
  }
];
```

- **src/index.css :**

```
:root {
  --primary: #7c3aed;
  --primary-dark: #6d28d9;
  --primary-light: #ede9fe;
  --secondary: #0ea5e9;
  --accent: #f59e0b;
  --bg-main: #f8fafc;
  --bg-card: #ffffff;
  --text-main: #0f172a;
  --text-muted: #64748b;
  --border-color: #e2e8f0;
  --radius: 12px;
  --font-family: 'Plus Jakarta Sans', system-ui, sans-serif;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: var(--font-family);
  background-color: var(--bg-main);
  color: var(--text-main);
  line-height: 1.6;
}

.app-wrapper {
  min-height: 100vh;
  display: flex;
```

```
  flex-direction: column;
}

/* Navbar */
.navbar {
  position: sticky;
  top: 0;
  z-index: 100;
  background: rgba(255, 255, 255, 0.95);
  backdrop-filter: blur(12px);
  border-bottom: 1px solid var(--border-color);
  padding: 0.9rem 2rem;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.nav-brand {
  display: flex;
  align-items: center;
  gap: 0.6rem;
  font-size: 1.25rem;
  font-weight: 800;
  color: var(--primary);
  text-decoration: none;
}

.nav-links {
  display: flex;
  gap: 0.5rem;
  list-style: none;
  align-items: center;
}

.nav-item a {
  text-decoration: none;
  color: var(--text-muted);
  font-weight: 600;
  font-size: 0.925rem;
  padding: 0.5rem 1rem;
  border-radius: 8px;
  transition: all 0.2s ease;
}

.nav-item a:hover {
  color: var(--primary);
  background: var(--primary-light);
}

.nav-item a.active {
  color: var(--primary-dark);
  background: var(--primary-light);
  font-weight: 700;
}

.main-content {
  flex: 1;
  max-width: 1200px;
  width: 100%;
  margin: 0 auto;
```

```
  padding: 2.5rem 1.5rem;
}

/* Hero Section */
.hero-section {
  text-align: center;
  padding: 3.5rem 1rem;
  background: linear-gradient(135deg, #f5f3ff 0%, #ede9fe 100%);
  border-radius: 18px;
  border: 1px solid #ddd6fe;
  margin-bottom: 3rem;
}

.hero-title {
  font-size: 2.75rem;
  font-weight: 800;
  color: #4c1d95;
  margin-bottom: 1rem;
  line-height: 1.2;
}

.hero-subtitle {
  font-size: 1.15rem;
  color: #6d28d9;
  max-width: 650px;
  margin: 0 auto 1.75rem;
}

.hero-buttons {
  display: flex;
  gap: 1rem;
  justify-content: center;
  flex-wrap: wrap;
}

/* Buttons */
.btn {
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
  font-size: 0.95rem;
  font-weight: 600;
  padding: 0.7rem 1.5rem;
  border-radius: 9px;
  text-decoration: none;
  border: none;
  cursor: pointer;
  transition: all 0.2s ease;
  font-family: inherit;
}

.btn-primary { background: var(--primary); color: white; }
.btn-primary:hover { background: var(--primary-dark); }
.btn-outline { background: white; color: var(--text-main); border: 1px solid var(--border-color);
}
.btn-outline:hover { background: #f1f5f9; }
.btn-sm { padding: 0.4rem 0.85rem; font-size: 0.85rem; }

/* Grid */
.grid-2 {
```

```
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(380px, 1fr));
  gap: 1.75rem;
}

.grid-3 {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(320px, 1fr));
  gap: 1.75rem;
}

/* Cards */
.card {
  background: var(--bg-card);
  border: 1px solid var(--border-color);
  border-radius: var(--radius);
  padding: 1.75rem;
  box-shadow: 0 1px 3px rgba(0,0,0,0.05);
  transition: transform 0.2s, box-shadow 0.2s;
  display: flex;
  flex-direction: column;
}

.card:hover {
  transform: translateY(-3px);
  box-shadow: 0 10px 20px rgba(0,0,0,0.08);
}

.badge {
  display: inline-block;
  padding: 0.25rem 0.75rem;
  border-radius: 9999px;
  font-size: 0.75rem;
  font-weight: 700;
  text-transform: uppercase;
}

.badge-purple { background: var(--primary-light); color: var(--primary); }
.badge-blue { background: #e0f2fe; color: #0369a1; }
.badge-amber { background: #fef3c7; color: #b45309; }

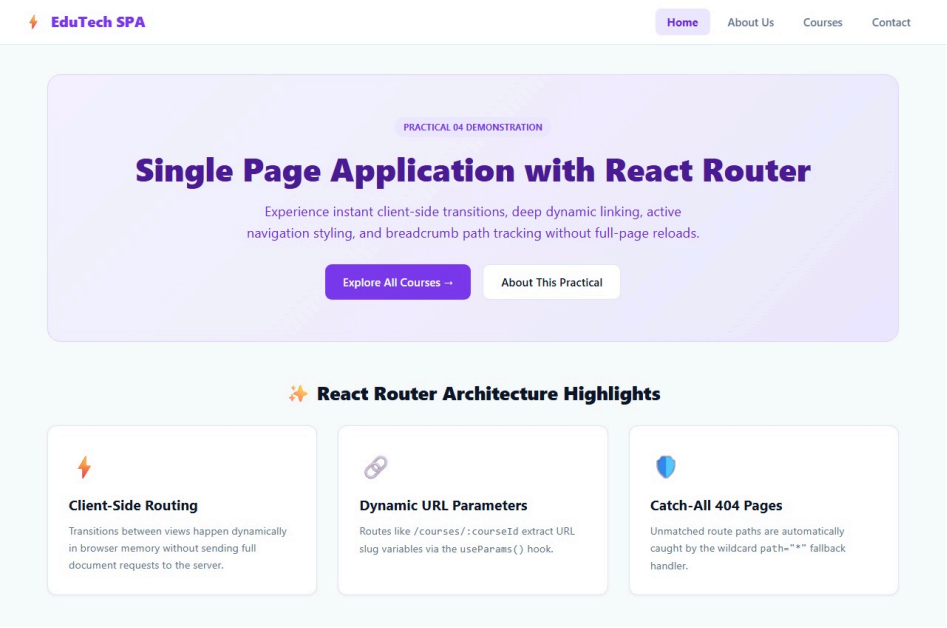
/* Breadcrumbs */
.breadcrumb {
  display: flex;
  align-items: center;
  gap: 0.5rem;
  font-size: 0.9rem;
  color: var(--text-muted);
  margin-bottom: 1.5rem;
}

.breadcrumb a {
  color: var(--primary);
  text-decoration: none;
  font-weight: 600;
}

.footer {
  background: #0f172a;
  color: #94a3b8;
```

```
  padding: 2rem 1.5rem;
  text-align: center;
  font-size: 0.9rem;
  border-top: 1px solid #1e293b;
  margin-top: auto;
}
```

4. Output :



Practical – 05 : Develop an Express.js Backend Server implementing Modular Routing, Custom Middleware, and Full RESTful CRUD Operations.

1. Commands :

```
mkdir MERN_05 && cd MERN_05
npm init -y
npm install express cors morgan
node server.js
```

2. Project structure :

```
MERN_05/
|-- controllers/
|   |-- productController.js
|-- routes/
|   |-- productRoutes.js
|-- middleware/
|   |-- logger.js
|   |-- errorHandler.js
|-- data/
|   |-- products.json
|-- server.js
|-- package.json
|-- test-requests.http
```

3. Source Code :

- **server.js :**

```
const express = require('express');
const cors = require('cors');
const requestLogger = require('./middleware/logger');
const errorHandler = require('./middleware/errorHandler');
const productRoutes = require('./routes/productRoutes');

const app = express();
const PORT = process.env.PORT || 5000;

// Built-in & Third-party Middleware
app.use(cors());
app.use(express.json()); // Parses application/json body
app.use(express.urlencoded({ extended: true }));
app.use(requestLogger);  // Custom Request Logger

// Welcome / Health Route
app.get('/', (req, res) => {
  res.status(200).json({
    success: true,
    message: 'Welcome to Practical 05: Express.js Routing & CRUD API Server',
    endpoints: {
      getAllProducts: 'GET /api/products',
      getProductById: 'GET /api/products/:id',
      createProduct: 'POST /api/products',
      updateProduct: 'PUT /api/products/:id',
      deleteProduct: 'DELETE /api/products/:id'
    },
    timestamp: new Date().toISOString()
  });
});
```

```
// Modular Routes Registration
app.use('/api/products', productRoutes);

// Fallback 404 Route Handler for undefined endpoints
app.use('*', (req, res) => {
  res.status(404).json({
    success: false,
    message: `Cannot ${req.method} ${req.originalUrl}. Route not found.`
  });
});

// Central Error Handler Middleware
app.use(errorHandler);

// Start HTTP Server
app.listen(PORT, () => {
  console.log(`================================================`);
  console.log(`🚀 Express CRUD Server is running on Port: ${PORT}`);
  console.log(`🔗 Local URL: http://localhost:${PORT}`);
  console.log(`📦 Products Endpoint: http://localhost:${PORT}/api/products`);
  console.log(`================================================`);
});
```

- **routes/productRoutes.js :**

```
const express = require('express');
const router = express.Router();
const {
  getAllProducts,
  getProductById,
  createProduct,
  updateProduct,
  deleteProduct
} = require('../controllers/productController');

// Define REST routes for /api/products
router.route('/')
  .get(getAllProducts)
  .post(createProduct);

router.route('/:id')
  .get(getProductById)
  .put(updateProduct)
  .delete(deleteProduct);

module.exports = router;
```

- **controllers/productController.js :**

```
const fs = require('fs');
const path = require('path');

const dataFilePath = path.join(__dirname, '..', 'data', 'products.json');
```

```
// Helper to read products
const readProducts = () => {
  try {
    const data = fs.readFileSync(dataFilePath, 'utf8');
    return JSON.parse(data);
  } catch (error) {
    return [];
  }
};

// Helper to save products
const saveProducts = (products) => {
  fs.writeFileSync(dataFilePath, JSON.stringify(products, null, 2), 'utf8');
};

/**
 * @desc   Get all products (with optional filtering & search)
 * @route  GET /api/products
 */
const getAllProducts = (req, res) => {
  let products = readProducts();
  const { category, search, minPrice, maxPrice } = req.query;

  // Filter by category
  if (category) {
    products = products.filter(p => p.category.toLowerCase() === category.toLowerCase());
  }

  // Search by name
  if (search) {
    products = products.filter(p => p.name.toLowerCase().includes(search.toLowerCase()));
  }

  // Filter by price range
  if (minPrice) {
    products = products.filter(p => p.price >= parseFloat(minPrice));
  }
  if (maxPrice) {
    products = products.filter(p => p.price <= parseFloat(maxPrice));
  }

  res.status(200).json({
    success: true,
    count: products.length,
    data: products
  });
};

/**
 * @desc   Get single product by ID
 * @route  GET /api/products/:id
 */
const getProductById = (req, res) => {
  const products = readProducts();
  const productId = parseInt(req.params.id);

  const product = products.find(p => p.id === productId);

  if (!product) {
```

```
    return res.status(404).json({
      success: false,
      message: `Product with ID ${req.params.id} was not found.`
    });
  }

  res.status(200).json({
    success: true,
    data: product
  });
};

/**
 * @desc   Create new product
 * @route  POST /api/products
 */
const createProduct = (req, res) => {
  const { name, category, price, stock } = req.body;

  // Validation
  if (!name || !category || price === undefined) {
    return res.status(400).json({
      success: false,
      message: 'Validation failed: "name", "category", and "price" are required fields.'
    });
  }

  if (isNaN(price) || price < 0) {
    return res.status(400).json({
      success: false,
      message: 'Validation failed: "price" must be a positive number.'
    });
  }

  const products = readProducts();

  const newProduct = {
    id: products.length > 0 ? Math.max(...products.map(p => p.id)) + 1 : 1,
    name: name.trim(),
    category: category.trim(),
    price: parseFloat(price),
    stock: stock !== undefined ? parseInt(stock) : 0,
    inStock: stock !== undefined ? parseInt(stock) > 0 : true,
    createdAt: new Date().toISOString()
  };

  products.push(newProduct);
  saveProducts(products);

  res.status(201).json({
    success: true,
    message: 'Product created successfully.',
    data: newProduct
  });
};

/**
 * @desc   Update product completely (PUT)
 * @route  PUT /api/products/:id
 */
```

```
const updateProduct = (req, res) => {
  const productId = parseInt(req.params.id);
  const { name, category, price, stock, inStock } = req.body;

  const products = readProducts();
  const index = products.findIndex(p => p.id === productId);

  if (index === -1) {
    return res.status(404).json({
      success: false,
      message: `Product with ID ${req.params.id} does not exist.`
    });
  }

  // Full update
  products[index] = {
    ...products[index],
    name: name !== undefined ? name.trim() : products[index].name,
    category: category !== undefined ? category.trim() : products[index].category,
    price: price !== undefined ? parseFloat(price) : products[index].price,
    stock: stock !== undefined ? parseInt(stock) : products[index].stock,
    inStock: inStock !== undefined ? inStock : (stock !== undefined ? parseInt(stock) > 0 :
products[index].inStock),
    updatedAt: new Date().toISOString()
  };

  saveProducts(products);

  res.status(200).json({
    success: true,
    message: 'Product updated successfully.',
    data: products[index]
  });
};

/**
 * @desc   Delete product by ID
 * @route  DELETE /api/products/:id
 */
const deleteProduct = (req, res) => {
  const productId = parseInt(req.params.id);
  const products = readProducts();
  const productToDelete = products.find(p => p.id === productId);

  if (!productToDelete) {
    return res.status(404).json({
      success: false,
      message: `Product with ID ${req.params.id} not found.`
    });
  }

  const updatedProducts = products.filter(p => p.id !== productId);
  saveProducts(updatedProducts);

  res.status(200).json({
    success: true,
    message: `Product "${productToDelete.name}" (ID: ${productId}) has been deleted.`,
    data: productToDelete
  });
};
```

```
module.exports = {
  getAllProducts,
  getProductById,
  createProduct,
  updateProduct,
  deleteProduct
};
```

- **middleware/logger.js :**

```
/**
 * Request Logger Middleware
 * Logs HTTP Method, Request Path, Status Code & Time Taken
 */
const requestLogger = (req, res, next) => {
  const start = Date.now();
  const timestamp = new Date().toISOString();

  res.on('finish', () => {
    const duration = Date.now() - start;
    console.log(`[${timestamp}] ${req.method} ${req.originalUrl} | Status: ${res.statusCode}
(${duration}ms)`);
  });

  next();
};

module.exports = requestLogger;
```

- **middleware/errorHandler.js :**

```
/**
 * Central Error Handling Middleware
 */
const errorHandler = (err, req, res, next) => {
  console.error('Server Error:', err.stack || err.message);

  const statusCode = res.statusCode === 200 ? 500 : res.statusCode;

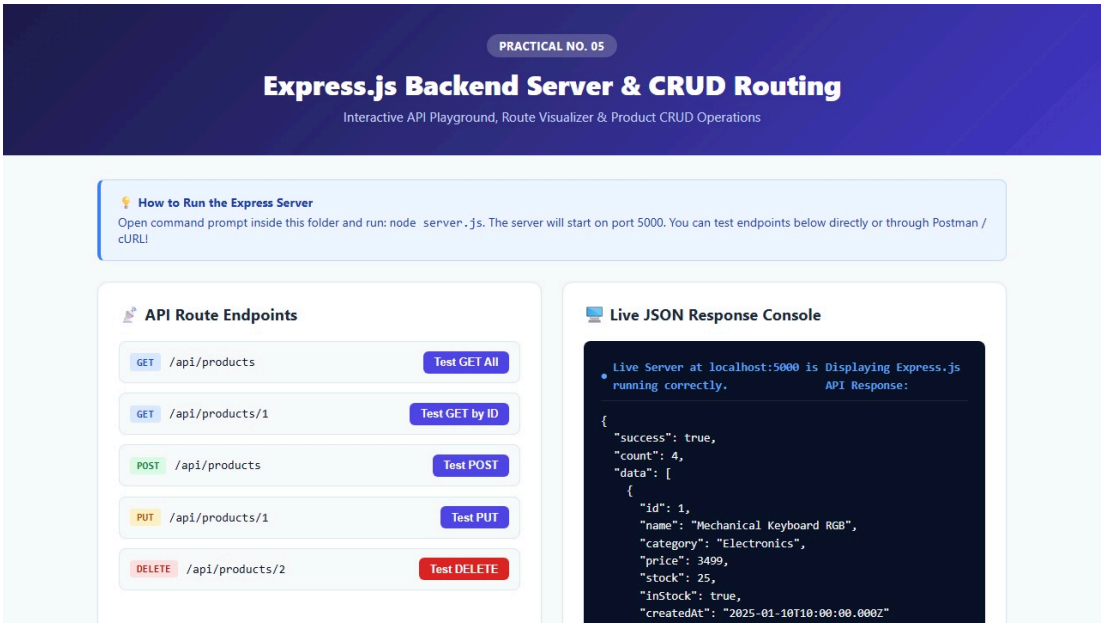
  res.status(statusCode).json({
    success: false,
    message: err.message || 'Internal Server Error',
    error: process.env.NODE_ENV === 'production' ? null : err.stack
  });
};

module.exports = errorHandler;
```

- **data/products.json :**

```
[
  {
    "id": 1,
    "name": "Mechanical Keyboard RGB",
    "category": "Electronics",
    "price": 3499,
    "stock": 25,
    "inStock": true,
    "createdAt": "2025-01-10T10:00:00.000Z"
  },
  {
    "id": 2,
    "name": "Wireless Ergonomic Mouse",
    "category": "Electronics",
    "price": 1799,
    "stock": 40,
    "inStock": true,
    "createdAt": "2025-01-12T11:30:00.000Z"
  },
  {
    "id": 3,
    "name": "27-inch 4K UHD Monitor",
    "category": "Displays",
    "price": 24999,
    "stock": 10,
    "inStock": true,
    "createdAt": "2025-01-15T14:15:00.000Z"
  },
  {
    "id": 4,
    "name": "Noise Cancelling Headphones",
    "category": "Audio",
    "price": 7999,
    "stock": 18,
    "inStock": true,
    "createdAt": "2025-01-18T09:45:00.000Z"
  }
]
```


4. Output :


PRACTICAL NO. 05
Express.js Backend Server & CRUD Routing
Interactive API Playground, Route Visualizer & Product CRUD Operations
How to Run the Express Server
Open command prompt inside this folder and run: node server.js. The server will start on port 5000. You can test endpoints below directly or through Postman / cURL!
API Route Endpoints
GET /api/products
Test GET All
GET /api/products/1
Test GET by ID
POST /api/products
Test POST
PUT /api/products/1
Test PUT
DELETE /api/products/2
Test DELETE
Live JSON Response Console
Live Server at localhost:5000 is running correctly.
API Response:
{
"success": true,
"count": 4,
"data": [
{
"id": 1,
"name": "Mechanical Keyboard RGB",
"category": "Electronics",
"price": 3499,
"stock": 25,
"inStock": true,
"createdAt": "2025-01-10T10:00:00.000Z"

Practical – 06 : Develop a Student Records REST API using Express.js with JSON Data Store, HTTP Status Codes, Validation Middleware, and Postman Collection.

1. Commands :

```
mkdir MERN_06 && cd MERN_06
npm init -y
npm install express cors morgan
node server.js
```

2. Project structure :

```
MERN_06/
|-- controllers/
|   |-- studentController.js
|-- routes/
|   |-- studentRoutes.js
|-- middleware/
|   |-- validator.js
|   |-- logger.js
|-- data/
|   |-- students.json
|-- Student_Records_API.postman_collection.json
|-- server.js
|-- package.json
```

3. Source Code :

- **server.js :**

```
const express = require('express');
const cors = require('cors');
const requestLogger = require('./middleware/logger');
const studentRoutes = require('./routes/studentRoutes');

const app = express();
const PORT = process.env.PORT || 5000;

app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(requestLogger);

// Home route
app.get('/', (req, res) => {
  res.status(200).json({
    success: true,
    message: 'Practical 06: RESTful APIs for Student Records Management',
    endpoints: {
      getAllStudents: 'GET /api/students',
      getStudentByIdOrRoll: 'GET /api/students/:id',
      createStudent: 'POST /api/students',
      updateStudent: 'PUT /api/students/:id',
      deleteStudent: 'DELETE /api/students/:id',
      getOverviewStats: 'GET /api/students/stats/overview'
    },
    postmanCollection: 'Student_Records_API.postman_collection.json'
  });
});
```

```
app.use('/api/students', studentRoutes);

// Fallback 404
app.use('*', (req, res) => {
  res.status(404).json({
    success: false,
    message: `Resource ${req.originalUrl} not found on this server.`
  });
});

app.listen(PORT, () => {
  console.log(`================================================`);
  console.log(`🚀 Practical 06: Student Records REST API Server`);
  console.log(`🌐 Server running at: http://localhost:${PORT}`);
  console.log(`📚 Base API Endpoint: http://localhost:${PORT}/api/students`);
  console.log(`📮 Postman Collection ready in project folder`);
  console.log(`================================================`);
});
```

- **routes/studentRoutes.js :**

```
const express = require('express');
const router = express.Router();
const {
  getAllStudents,
  getStudentById,
  createStudent,
  updateStudent,
  deleteStudent,
  getStudentStats
} = require('../controllers/studentController');
const { validateStudentPayload } = require('../middleware/validator');

// Analytics / Stats endpoint (placed before :id route)
router.get('/stats/overview', getStudentStats);

// Main Student REST endpoints
router.route('/')
  .get(getAllStudents)
  .post(validateStudentPayload, createStudent);

router.route('/:id')
  .get(getStudentById)
  .put(validateStudentPayload, updateStudent)
  .delete(deleteStudent);

module.exports = router;
```

- **controllers/studentController.js :**

```
const fs = require('fs');
const path = require('path');

const dataFilePath = path.join(__dirname, '..', 'data', 'students.json');
```

```
const getStudentsData = () => {
  try {
    const raw = fs.readFileSync(dataFilePath, 'utf8');
    return JSON.parse(raw);
  } catch (err) {
    return [];
  }
};

const saveStudentsData = (data) => {
  fs.writeFileSync(dataFilePath, JSON.stringify(data, null, 2), 'utf8');
};

/**
 * @desc   Get All Students with search, filtering, sorting, pagination
 * @route  GET /api/students
 */
const getAllStudents = (req, res) => {
  let students = getStudentsData();
  const { department, semester, minCgpa, search, sortBy } = req.query;

  // Filter by department
  if (department) {
    students = students.filter(s => s.department.toLowerCase() === department.toLowerCase());
  }

  // Filter by semester
  if (semester) {
    students = students.filter(s => s.semester === parseInt(semester));
  }

  // Filter by minimum CGPA
  if (minCgpa) {
    students = students.filter(s => s.cgpa >= parseFloat(minCgpa));
  }

  // Search by keyword (name, rollNo, email)
  if (search) {
    const q = search.toLowerCase();
    students = students.filter(s =>
      s.name.toLowerCase().includes(q) ||
      s.rollNo.toLowerCase().includes(q) ||
      s.email.toLowerCase().includes(q)
    );
  }

  // Sort by cgpa or name
  if (sortBy === 'cgpa_desc') {
    students.sort((a, b) => b.cgpa - a.cgpa);
  } else if (sortBy === 'name') {
    students.sort((a, b) => a.name.localeCompare(b.name));
  }

  res.status(200).json({
    success: true,
    totalRecords: students.length,
    data: students
  });
};
```

```
/**
 * @desc   Get Student By ID or Roll Number
 * @route  GET /api/students/:id
 */
const getStudentById = (req, res) => {
  const students = getStudentsData();
  const param = req.params.id;

  const student = students.find(s => s.id.toString() === param || s.rollNo.toLowerCase() ===
param.toLowerCase());

  if (!student) {
    return res.status(404).json({
      success: false,
      message: `Student with ID or Roll No "${param}" was not found.`
    });
  }

  res.status(200).json({
    success: true,
    data: student
  });
};

/**
 * @desc   Create New Student Record
 * @route  POST /api/students
 */
const createStudent = (req, res) => {
  const students = getStudentsData();
  const { rollNo, name, email, phone, department, semester, cgpa } = req.body;

  // Check unique rollNo
  const existingRoll = students.find(s => s.rollNo.toLowerCase() ===
rollNo.trim().toLowerCase());
  if (existingRoll) {
    return res.status(409).json({
      success: false,
      message: `Student with Roll Number "${rollNo}" already exists in records.`
    });
  }

  // Check unique email
  const existingEmail = students.find(s => s.email.toLowerCase() === email.trim().toLowerCase());
  if (existingEmail) {
    return res.status(409).json({
      success: false,
      message: `Student with Email "${email}" is already registered.`
    });
  }

  const nextId = students.length > 0 ? Math.max(...students.map(s => s.id)) + 1 : 1;

  const newStudent = {
    id: nextId,
    rollNo: rollNo.trim().toUpperCase(),
    name: name.trim(),
    email: email.trim().toLowerCase(),
    phone: phone ? phone.trim() : 'N/A',
```

```
    department: department.trim(),
    semester: parseInt(semester),
    cgpa: cgpa !== undefined ? parseFloat(cgpa) : 0.0,
    enrolledDate: new Date().toISOString().split('T')[0]
  };

  students.push(newStudent);
  saveStudentsData(students);

  res.status(201).json({
    success: true,
    message: 'Student record created successfully.',
    data: newStudent
  });
};

/**
 * @desc   Update Student Record
 * @route  PUT /api/students/:id
 */
const updateStudent = (req, res) => {
  const students = getStudentsData();
  const param = req.params.id;
  const index = students.findIndex(s => s.id.toString() === param || s.rollNo.toLowerCase() ===
param.toLowerCase());

  if (index === -1) {
    return res.status(404).json({
      success: false,
      message: `Student record "${param}" not found for update.`
    });
  }

  const { name, email, phone, department, semester, cgpa } = req.body;

  students[index] = {
    ...students[index],
    name: name !== undefined ? name.trim() : students[index].name,
    email: email !== undefined ? email.trim().toLowerCase() : students[index].email,
    phone: phone !== undefined ? phone.trim() : students[index].phone,
    department: department !== undefined ? department.trim() : students[index].department,
    semester: semester !== undefined ? parseInt(semester) : students[index].semester,
    cgpa: cgpa !== undefined ? parseFloat(cgpa) : students[index].cgpa,
    updatedAt: new Date().toISOString()
  };

  saveStudentsData(students);

  res.status(200).json({
    success: true,
    message: 'Student record updated successfully.',
    data: students[index]
  });
};

/**
 * @desc   Delete Student Record
 * @route  DELETE /api/students/:id
 */
const deleteStudent = (req, res) => {
```

```
  const students = getStudentsData();
  const param = req.params.id;
  const student = students.find(s => s.id.toString() === param || s.rollNo.toLowerCase() ===
param.toLowerCase());

  if (!student) {
    return res.status(404).json({
      success: false,
      message: `Student record "${param}" not found for deletion.`
    });
  }

  const filtered = students.filter(s => s.id !== student.id);
  saveStudentsData(filtered);

  res.status(200).json({
    success: true,
    message: `Student record for "${student.name}" (Roll: ${student.rollNo}) deleted
successfully.`,
    data: student
  });
};

/**
 * @desc   Get Department-wise summary & statistics
 * @route  GET /api/students/stats/overview
 */
const getStudentStats = (req, res) => {
  const students = getStudentsData();

  const total = students.length;
  const deptCounts = {};
  let totalCgpa = 0;

  students.forEach(s => {
    deptCounts[s.department] = (deptCounts[s.department] || 0) + 1;
    totalCgpa += s.cgpa || 0;
  });

  const avgCgpa = total > 0 ? (totalCgpa / total).toFixed(2) : 0;

  res.status(200).json({
    success: true,
    statistics: {
      totalStudents: total,
      averageCgpa: parseFloat(avgCgpa),
      departmentDistribution: deptCounts
    }
  });
};

module.exports = {
  getAllStudents,
  getStudentById,
  createStudent,
  updateStudent,
  deleteStudent,
  getStudentStats
};
```

- **middleware/validator.js :**

```
/**
 * Validation Middleware for Student Record Creation and Update
 */
const validateStudentPayload = (req, res, next) => {
  const { rollNo, name, email, department, semester, cgpa } = req.body;
  const errors = [];

  // Required fields for POST
  if (req.method === 'POST') {
    if (!rollNo || !rollNo.toString().trim()) errors.push('Field "rollNo" is required.');
    if (!name || !name.trim()) errors.push('Field "name" is required.');
    if (!email || !email.trim()) errors.push('Field "email" is required.');
    if (!department || !department.trim()) errors.push('Field "department" is required.');
    if (semester === undefined) errors.push('Field "semester" is required.');
  }

  // Email format check if provided
  if (email) {
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailRegex.test(email.trim())) {
      errors.push('Field "email" must be a valid email format.');
    }
  }

  // Semester range check if provided
  if (semester !== undefined) {
    const semNum = parseInt(semester);
    if (isNaN(semNum) || semNum < 1 || semNum > 8) {
      errors.push('Field "semester" must be an integer between 1 and 8.');
    }
  }

  // CGPA range check if provided
  if (cgpa !== undefined) {
    const cgpaNum = parseFloat(cgpa);
    if (isNaN(cgpaNum) || cgpaNum < 0 || cgpaNum > 10) {
      errors.push('Field "cgpa" must be a float between 0.00 and 10.00.');
    }
  }

  if (errors.length > 0) {
    return res.status(400).json({
      success: false,
      message: 'Validation failed on student record payload.',
      errors
    });
  }

  next();
};

module.exports = { validateStudentPayload };
```

- **data/students.json :**

```
[
  {
    "id": 1,
    "rollNo": "21CS001",
    "name":  Prince Prajapati
    "email":  prince@college.edu,
    "phone": "9876543210",
    "department": "Computer Engineering",
    "semester": 6,
    "cgpa": 8.95,
    "enrolledDate": "2022-08-15"
  },
  {
    "id": 2,
    "rollNo": "21IT042",
    "name": "Aarav Patel",
    "email": "aarav.patel@college.edu",
    "phone": "9823456781",
    "department": "Information Technology",
    "semester": 6,
    "cgpa": 9.12,
    "enrolledDate": "2022-08-15"
  },
  {
    "id": 3,
    "rollNo": "22AI015",
    "name": "Sneha Roy",
    "email": "sneha.roy@college.edu",
    "phone": "9734567890",
    "department": "AI & Data Science",
    "semester": 4,
    "cgpa": 8.70,
    "enrolledDate": "2023-08-10"
  },
  {
    "id": 4,
    "rollNo": "21EC028",
    "name": "Rohan Deshmukh",
    "email": "rohan.d@college.edu",
    "phone": "9845678901",
    "department": "Electronics & Comm",
    "semester": 6,
    "cgpa": 7.85,
    "enrolledDate": "2022-08-15"
  }
]
```


4. Output :

PRACTICAL NO. 06

Student Records RESTful API with Postman Testing

REST API Endpoints, Status Codes & Postman Test Collection Integration

Postman Test Collection Available

Import `Student_Records_API.postman_collection.json` in your Postman desktop client to execute the entire 11-step REST API test suite!

RESTful Endpoints

GET	/api/students	Test GET All
GET	/api/students/26084231038	Test by Roll No
GET	/api/students/stats/overview	Test Stats API
POST	/api/students	Test POST Student
DELETE	/api/students/21CS099	Test DELETE

Response Output

```
Live Server at localhost:5000 is connected.   Express REST API Response:

{
  "id": 1,
  "rollNo": "26084231038",
  "name": "Prince Prajapati",
  "email": "prince@college.edu",
  "department": "Computer Engineering",
  "semester": 6,
  "cgpa": 8.95
}

Tip: Run 'node server.js' in terminal to start the live Node.js REST server!
```

Practical – 07 : Develop a complete Employee Management RESTful API using Express.js and MongoDB (Mongoose ODM) with Schema Validation and Aggregations.

1. Commands :

```
mkdir MERN_07 && cd MERN_07
npm init -y
npm install express mongoose dotenv cors morgan
node seed.js
node server.js
```

2. Project structure :

```
MERN_07/
|-- config/
|   |-- db.js
|-- models/
|   |-- Employee.js
|-- controllers/
|   |-- employeeController.js
|-- routes/
|   |-- employeeRoutes.js
|-- .env
|-- seed.js
|-- server.js
|-- package.json
```

3. Source Code :

- **server.js :**

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const connectDB = require('./config/db');
const employeeRoutes = require('./routes/employeeRoutes');

const app = express();
const PORT = process.env.PORT || 5000;

// Connect to MongoDB Atlas
connectDB();

// Middlewares
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Request Logger
app.use((req, res, next) => {
  const start = Date.now();
  res.on('finish', () => {
    console.log(`[MongoDB API] ${req.method} ${req.originalUrl} - ${res.statusCode} (${Date.now()
- start}ms)`);
  });
  next();
});

// Welcome / Health Route
```

```
app.get('/', (req, res) => {
  res.status(200).json({
    success: true,
    message: 'Practical 07: Employee Management API with Express & MongoDB Atlas',
    endpoints: {
      getAllEmployees: 'GET /api/employees',
      getEmployeeByIdOrEmpId: 'GET /api/employees/:id',
      createEmployee: 'POST /api/employees',
      updateEmployee: 'PUT /api/employees/:id',
      deleteEmployee: 'DELETE /api/employees/:id',
      departmentAnalytics: 'GET /api/employees/stats/summary'
    }
  });
});

// Mount Routes
app.use('/api/employees', employeeRoutes);

// Fallback 404
app.use('*', (req, res) => {
  res.status(404).json({
    success: false,
    message: `Route ${req.originalUrl} not found on this Employee Management API.`
  });
});

// Global Error Handler
app.use((err, req, res, next) => {
  console.error('Unhandled Error:', err.message);
  res.status(500).json({
    success: false,
    message: err.message || 'Internal Server Error'
  });
});

app.listen(PORT, () => {
  console.log(`================================================`);
  console.log(`🚀 Practical 07: Employee Management API`);
  console.log(`🌐 Server running at: http://localhost:${PORT}`);
  console.log(`📡 Endpoints at: http://localhost:${PORT}/api/employees`);
  console.log(`🔗 MongoDB URI: ${process.env.MONGO_URI ? 'Loaded from .env' : 'Not Provided'}`);
  console.log(`================================================`);
});
```

- **config/db.js :**

```
const mongoose = require('mongoose');

/**
 * Connect to MongoDB / MongoDB Atlas using Mongoose
 */
const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI, {
      serverSelectionTimeoutMS: 5000 // 5 seconds timeout
    });
```

```
    console.log(`✅MongoDB Connected Successfully: ${conn.connection.host}`);
    console.log(`📂 Database Name: ${conn.connection.name}`);
  } catch (error) {
    console.error(`❌MongoDB Connection Error: ${error.message}`);
    console.log(`💡 Tip: Make sure your MongoDB Atlas connection string in .env is correct and
your IP address is whitelisted in Atlas Network Access.`);
  }
};

module.exports = connectDB;
```

- **models/Employee.js :**

```
const mongoose = require('mongoose');

const employeeSchema = new mongoose.Schema(
  {
    empId: {
      type: String,
      required: [true, 'Employee ID is required.'],
      unique: true,
      trim: true,
      uppercase: true
    },
    firstName: {
      type: String,
      required: [true, 'First Name is required.'],
      trim: true,
      minlength: [2, 'First Name must be at least 2 characters.']
    },
    lastName: {
      type: String,
      required: [true, 'Last Name is required.'],
      trim: true
    },
    email: {
      type: String,
      required: [true, 'Email address is required.'],
      unique: true,
      trim: true,
      lowercase: true,
      match: [/^[^\s@]+@[^\s@]+\.[^\s@]+$/, 'Please provide a valid email address.']
    },
    phone: {
      type: String,
      required: [true, 'Phone number is required.'],
      trim: true
    },
    department: {
      type: String,
      required: [true, 'Department is required.'],
      enum: {
        values: ['Engineering', 'Human Resources', 'Finance', 'Marketing', 'Product',
'Operations'],
        message: '{VALUE} is not a supported department.'
      }
    },
```

```
    designation: {
      type: String,
      required: [true, 'Designation / Job Title is required.'],
      trim: true
    },
    salary: {
      type: Number,
      required: [true, 'Salary is required.'],
      min: [0, 'Salary cannot be negative.']
    },
    joiningDate: {
      type: Date,
      default: Date.now
    },
    status: {
      type: String,
      enum: ['Active', 'Inactive', 'On Leave'],
      default: 'Active'
    }
  },
  {
    timestamps: true, // Automatically manages createdAt and updatedAt
    toJSON: { virtuals: true },
    toObject: { virtuals: true }
  }
);

// Virtual property for full name
employeeSchema.virtual('fullName').get(function() {
  return `${this.firstName} ${this.lastName}`;
});

const Employee = mongoose.model('Employee', employeeSchema);

module.exports = Employee;
```

- **controllers/employeeController.js :**

```
const Employee = require('../models/Employee');

/**
 * @desc   Get All Employees with filtering, search, sorting & pagination
 * @route  GET /api/employees
 */
const getAllEmployees = async (req, res) => {
  try {
    const { department, status, search, minSalary, maxSalary, sortBy, page = 1, limit = 10 } =
req.query;

    const query = {};

    // Filter by department
    if (department) {
      query.department = department;
    }

    // Filter by status
```

```
    if (status) {
      query.status = status;
    }

    // Filter by salary range
    if (minSalary || maxSalary) {
      query.salary = {};
      if (minSalary) query.salary.$gte = Number(minSalary);
      if (maxSalary) query.salary.$lte = Number(maxSalary);
    }

    // Search keyword across firstName, lastName, email, designation
    if (search) {
      query.$or = [
        { firstName: { $regex: search, $options: 'i' } },
        { lastName: { $regex: search, $options: 'i' } },
        { email: { $regex: search, $options: 'i' } },
        { designation: { $regex: search, $options: 'i' } },
        { empId: { $regex: search, $options: 'i' } }
      ];
    }

    // Sorting
    let sortOption = { createdAt: -1 };
    if (sortBy === 'salary_desc') sortOption = { salary: -1 };
    else if (sortBy === 'salary_asc') sortOption = { salary: 1 };
    else if (sortBy === 'name') sortOption = { firstName: 1 };

    // Pagination
    const skip = (Number(page) - 1) * Number(limit);
    const totalRecords = await Employee.countDocuments(query);
    const employees = await Employee.find(query)
      .sort(sortOption)
      .skip(skip)
      .limit(Number(limit));

    res.status(200).json({
      success: true,
      count: employees.length,
      totalRecords,
      totalPages: Math.ceil(totalRecords / Number(limit)),
      currentPage: Number(page),
      data: employees
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: 'Failed to retrieve employees.',
      error: error.message
    });
  }
};

/**
 * @desc   Get Single Employee by MongoDB _id OR custom empId
 * @route  GET /api/employees/:id
 */
const getEmployeeById = async (req, res) => {
  try {
    const { id } = req.params;
```

```
    // Search by ObjectId or custom empId (e.g., EMP101)
    let employee;
    if (id.match(/^[0-9a-fA-F]{24}$/)) {
      employee = await Employee.findById(id);
    } else {
      employee = await Employee.findOne({ empId: id.toUpperCase() });
    }

    if (!employee) {
      return res.status(404).json({
        success: false,
        message: `Employee "${id}" was not found.`
      });
    }

    res.status(200).json({
      success: true,
      data: employee
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: 'Error fetching employee details.',
      error: error.message
    });
  }
};

/**
 * @desc   Create New Employee Record in MongoDB
 * @route  POST /api/employees
 */
const createEmployee = async (req, res) => {
  try {
    const employee = await Employee.create(req.body);

    res.status(201).json({
      success: true,
      message: 'Employee registered successfully in MongoDB Atlas.',
      data: employee
    });
  } catch (error) {
    // Handle Duplicate Key Error
    if (error.code === 11000) {
      const field = Object.keys(error.keyPattern)[0];
      return res.status(409).json({
        success: false,
        message: `Duplicate entry error: An employee with this ${field} already exists.`
      });
    }

    // Handle Mongoose Validation Errors
    if (error.name === 'ValidationError') {
      const messages = Object.values(error.errors).map(val => val.message);
      return res.status(400).json({
        success: false,
        message: 'Validation failed on employee fields.',
        errors: messages
      });
```

```
    }

    res.status(500).json({
      success: false,
      message: 'Failed to create employee.',
      error: error.message
    });
  }
};

/**
 * @desc   Update Employee Details in MongoDB
 * @route  PUT /api/employees/:id
 */
const updateEmployee = async (req, res) => {
  try {
    const { id } = req.params;
    let query = {};
    if (id.match(/^[0-9a-fA-F]{24}$/)) {
      query = { _id: id };
    } else {
      query = { empId: id.toUpperCase() };
    }

    const updatedEmployee = await Employee.findOneAndUpdate(
      query,
      req.body,
      { new: true, runValidators: true }
    );

    if (!updatedEmployee) {
      return res.status(404).json({
        success: false,
        message: `Employee "${id}" not found for update.`
      });
    }

    res.status(200).json({
      success: true,
      message: 'Employee record updated successfully.',
      data: updatedEmployee
    });
  } catch (error) {
    if (error.name === 'ValidationError') {
      const messages = Object.values(error.errors).map(val => val.message);
      return res.status(400).json({
        success: false,
        message: 'Validation failed on update.',
        errors: messages
      });
    }

    res.status(500).json({
      success: false,
      message: 'Failed to update employee.',
      error: error.message
    });
  }
};
```

```
/**
 * @desc   Delete Employee from MongoDB
 * @route  DELETE /api/employees/:id
 */
const deleteEmployee = async (req, res) => {
  try {
    const { id } = req.params;
    let query = {};
    if (id.match(/^[0-9a-fA-F]{24}$/)) {
      query = { _id: id };
    } else {
      query = { empId: id.toUpperCase() };
    }

    const deletedEmployee = await Employee.findOneAndDelete(query);

    if (!deletedEmployee) {
      return res.status(404).json({
        success: false,
        message: `Employee "${id}" not found for deletion.`
      });
    }

    res.status(200).json({
      success: true,
      message: `Employee "${deletedEmployee.firstName} ${deletedEmployee.lastName}"
(${deletedEmployee.empId}) deleted successfully.`,
      data: deletedEmployee
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: 'Failed to delete employee.',
      error: error.message
    });
  }
};

/**
 * @desc   Department Aggregate Metrics & Analytics (MongoDB Aggregation Pipeline)
 * @route  GET /api/employees/stats/summary
 */
const getEmployeeSummaryStats = async (req, res) => {
  try {
    const stats = await Employee.aggregate([
      {
        $group: {
          _id: '$department',
          totalEmployees: { $sum: 1 },
          avgSalary: { $avg: '$salary' },
          minSalary: { $min: '$salary' },
          maxSalary: { $max: '$salary' }
        }
      },
      {
        $project: {
          department: '$_id',
          totalEmployees: 1,
          avgSalary: { $round: ['$avgSalary', 2] },
          minSalary: 1,
```

```
          maxSalary: 1,
          _id: 0
        }
      },
      { $sort: { totalEmployees: -1 } }
    ]);

    const totalHeadcount = await Employee.countDocuments();

    res.status(200).json({
      success: true,
      totalHeadcount,
      departmentStats: stats
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: 'Failed to compute employee summary statistics.',
      error: error.message
    });
  }
};

module.exports = {
  getAllEmployees,
  getEmployeeById,
  createEmployee,
  updateEmployee,
  deleteEmployee,
  getEmployeeSummaryStats
};
```

- **routes/employeeRoutes.js :**

```
const express = require('express');
const router = express.Router();
const {
  getAllEmployees,
  getEmployeeById,
  createEmployee,
  updateEmployee,
  deleteEmployee,
  getEmployeeSummaryStats
} = require('../controllers/employeeController');

// Aggregate statistics route (Must be before /:id)
router.get('/stats/summary', getEmployeeSummaryStats);

// Main Employee routes
router.route('/')
  .get(getAllEmployees)
  .post(createEmployee);

router.route('/:id')
  .get(getEmployeeById)
  .put(updateEmployee)
  .delete(deleteEmployee);
```

```
module.exports = router;
```

- **seed.js :**

```
require('dotenv').config();
const mongoose = require('mongoose');
const Employee = require('./models/Employee');

const sampleEmployees = [
  {
    empId: 'EMP101',
    firstName: 'Prince'
    lastName: 'Mansuri',
    email: 'prince.prajapati@company.com'
    phone: '9876543210',
    department: 'Engineering',
    designation: 'Lead Full Stack Engineer',
    salary: 85000,
    status: 'Active'
  },
  {
    empId: 'EMP102',
    firstName: 'Ananya',
    lastName: 'Sharma',
    email: 'ananya.sharma@company.com',
    phone: '9812345678',
    department: 'Product',
    designation: 'Senior Product Manager',
    salary: 95000,
    status: 'Active'
  },
  {
    empId: 'EMP103',
    firstName: 'Vikram',
    lastName: 'Malhotra',
    email: 'vikram.m@company.com',
    phone: '9765432109',
    department: 'Engineering',
    designation: 'Backend Architect',
    salary: 110000,
    status: 'Active'
  },
  {
    empId: 'EMP104',
    firstName: 'Pooja',
    lastName: 'Desai',
    email: 'pooja.desai@company.com',
    phone: '9845678901',
    department: 'Human Resources',
    designation: 'HR Specialist',
    salary: 55000,
    status: 'Active'
  },
  {
    empId: 'EMP105',
    firstName: 'Rohan',
```

```
    lastName: 'Verma',
    email: 'rohan.v@company.com',
    phone: '9834567812',
    department: 'Finance',
    designation: 'Financial Analyst',
    salary: 65000,
    status: 'On Leave'
  }
];

const seedDatabase = async () => {
  try {
    console.log('Connecting to MongoDB database...');
    await mongoose.connect(process.env.MONGO_URI);
    console.log('✅Connected to MongoDB.');

    console.log('Clearing existing employee documents...');
    await Employee.deleteMany({});

    console.log('Inserting sample employee records...');
    await Employee.insertMany(sampleEmployees);

    console.log(`🌱 Successfully seeded ${sampleEmployees.length} employees into MongoDB!`);
    process.exit(0);
  } catch (error) {
    console.error('❌Seeder Failed:', error.message);
    process.exit(1);
  }
};

seedDatabase();
```

- **.env :**

```
# Server Configuration
PORT=5000
NODE_ENV=development

# MongoDB Atlas Connection URI
# Replace with your actual MongoDB Atlas cluster connection string:
# Format:
mongodb+srv://<username>:<password>@cluster0.xxxxx.mongodb.net/<dbname>?retryWrites=true&w=majority
MONGO_URI=mongodb+srv://admin:admin123@cluster0.mongodb.net/employee_db?retryWrites=true&w=majority
```

4. Output :

PRACTICAL NO. 07

Employee Management API with MongoDB Atlas

Mongoose ODM, Schema Validation, Aggregation Analytics & Cloud Database Integration

MongoDB Atlas Integration Details

Configure your MongoDB Atlas cluster connection string in .env. Run `npm run seed` to populate sample records, and node `server.js` to start the live server.

Employee API Endpoints

GET	/api/employees	Test GET All
GET	/api/employees/EMP101	Test by empId
GET	/api/employees/stats/summary	Test Aggregation
POST	/api/employees	Test POST
DELETE	/api/employees/EMP106	Test DELETE

Response Output

Live MongoDB Driver at localhost:5000 is connected. Connected to MongoDB Atlas: employee_db


```
{
  "success": true,
  "totalHeadcount": 5,
  "departmentStats": [
    {
      "department": "Engineering",
      "totalEmployees": 2,
      "avgSalary": 97500
    },
    {
      "department": "Product",
      "totalEmployees": 1,
      "avgSalary": 95000
```